

AUTONOMOUS DRIVING



50



**Design Of Autonomous
System
Group-3**

DESIGN OF AUTONOMOUS SYSTEMS

**Kingston
University
London**

**School of Engineering Department of Mechanical
Engineering**

Coursework Assessment Brief

Module Code	ME7027
Module Title	Design of Autonomous Systems
Title of Assessment	Development and Comparison of Autonomous Navigation Systems using Fuzzy Logic, Computer Vision, and Deep Learning
Assessment set by (and contact)	Dr Olga Duran and Esfandiar Khaleghi

Group Member Information	
K Number	Name
K2445563	: JOSHVA JONATHAN JOSEPH
K2447273	: RAHUL R MENON
K2441768	: OMKAR JOSHI
K2454461	: SARATH KUMAR KOMATHUKATTIL

DESIGN OF AUTONOMOUS SYSTEMS

Table of Contents

1.1. Platform Description	4
1.2. Sensor characterisation	9
1.3. Sensor fusion	13
1.4. Discussion: Platform Suitability for Autonomous Navigation.....	15
Part 2: Computer Vision.....	16
2.1 Object detection and identification.....	16
2.2 Lane Detection and Steering Control on LEGO EV3 Robot	23
Part 3: Deep Learning-Based Object Detection	25
3.1 Dataset Acquisition & Organisation	26
3.2 Neural Network Model Implementation and Evaluation	29
3.3 Data Augmentation for Performance Improvement	35
3.4 Hyperparameter Tuning	35
3.5 Final Testing and Evaluation	37
Part 4: Navigation control strategies.....	37
4.1 Fuzzy Logic Controller for Navigation Decisions	38
4.2 Lane Following and Autonomous Control.....	38
Part 5: Mechanical Design of Camera Mount.....	40
5.1 Problem Statement	40
5.2 Summary of Key Design Specifications.....	41
5.3 Conceptual Design Generation	42
5.4 Design Evaluation and Selection	47
5.5 Detailed Design (CAD).....	48
5.6 Design Communication	49
5.7 Prototype Development	50
Appendix 1, Sensor characterisation	52
Appendix 2, CAD Models	55
Appendix 3, Constrained Models	58
Appendix 4, Quencer Virtual Simulation Lab and Simulink	62
References	65

DESIGN OF AUTONOMOUS SYSTEMS

Part 1: Sensor Fusion & Actuator Characterisation

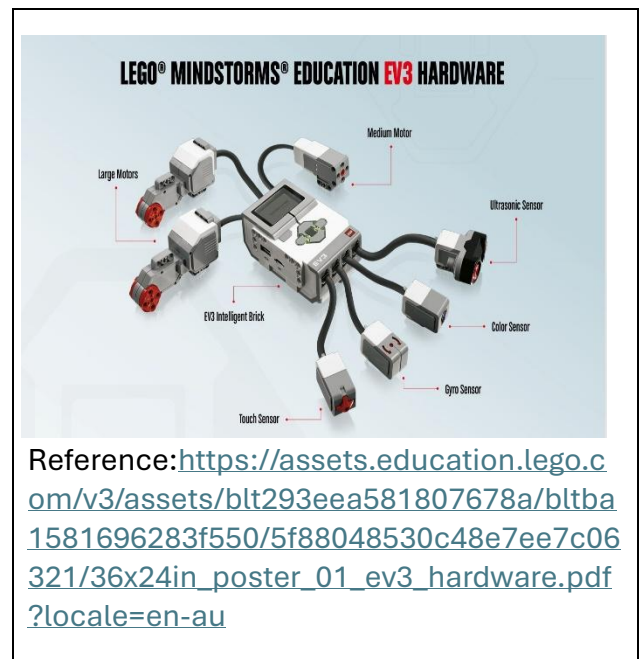
Autonomous navigation relies heavily on integrating multiple sensors and actuators to perceive the environment, localise the robot, and execute movement. Understanding the capabilities and limitations of each hardware component is essential for designing intelligent control systems.

This section provides a detailed description and characterisation of the sensors and actuators used in both the Lego EV3 robot and the Quanser QCar platforms. It also explores sensor fusion techniques for improving perception accuracy in dynamic environments, supporting tasks such as obstacle avoidance, lane detection, and stop sign recognition.

1.1. Platform Description

LEGO EV3

The LEGO EV3 robot is a compact, modular mobile platform. It is widely used for teaching the fundamentals of robotics, control systems, and sensor integration. The system comprises a programmable EV3 "brick" controller, a range of onboard sensors, and motorized actuators, making it a cost-effective solution for developing basic autonomous behaviours.



The **Sensors** used in the Lego EV3 for its autonomous behaviours are

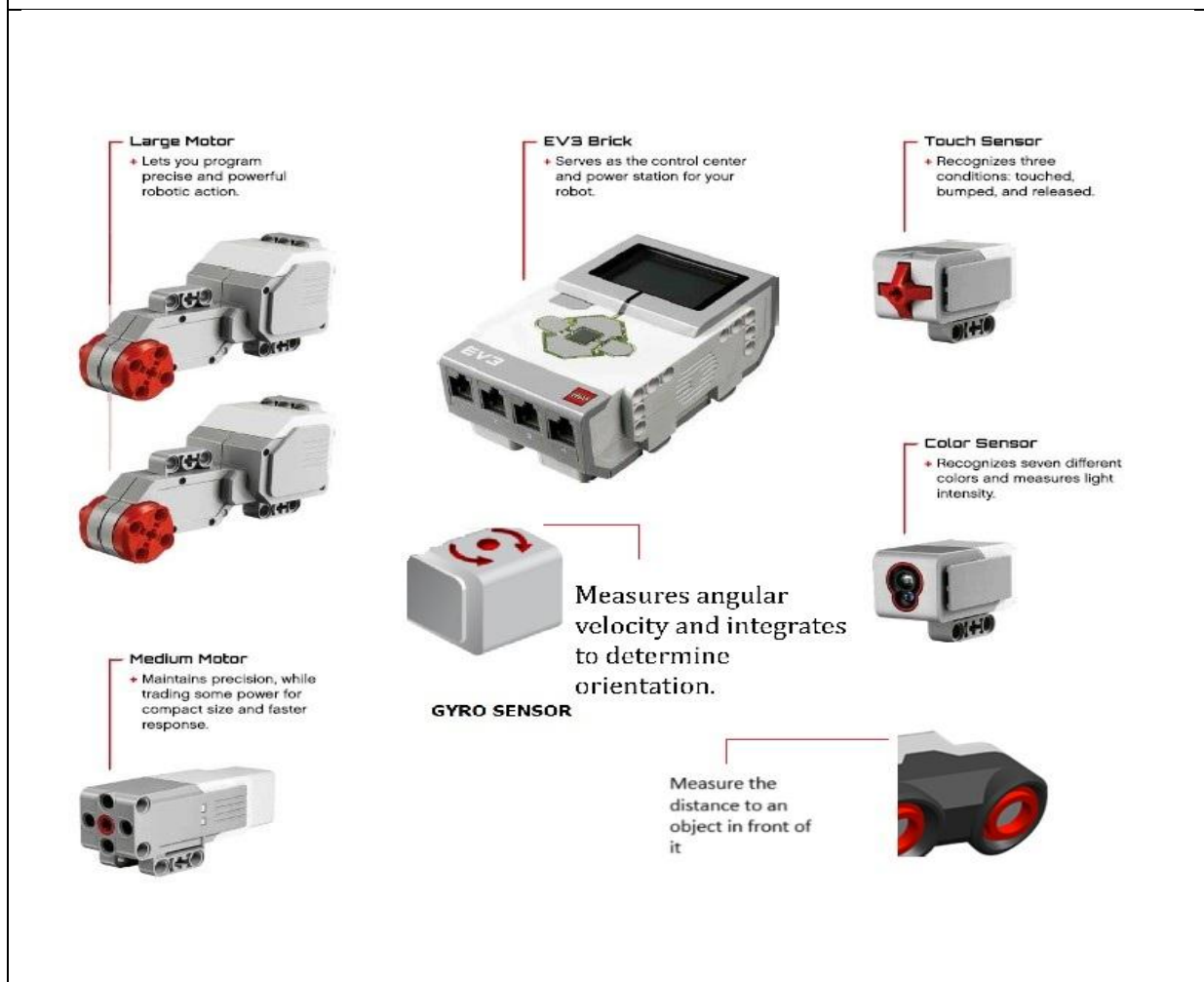
Sensor	Type	Operation and Use	Key Specifications
Ultrasonic Sensor	Distance	Emits ultrasonic pulses and measures echo time to estimate distance. Used for obstacle detection.	- Range: 3 cm to 250 cm - Accuracy: ± 1 cm
Touch Sensor	Binary Contact	Recognises three conditions: touched, bumped, and released. Used for bump detection or trigger events.	- Binary states - Debounced signal - Mechanical button

DESIGN OF AUTONOMOUS SYSTEMS

Gyro Sensor	Orientation	The Gyro Sensor detects rotational motion of the Robo	- Resolution: $\pm 1^\circ$ - Drift over time (up to $3^\circ/\text{min}$) - Range: $\pm 440^\circ/\text{sec}$
Colour Sensor	Reflectance / Colour	Recognises seven different colours and measures light intensity. Detects surface reflectivity and colour. Used for line-following and object recognition.	- Colour mode (7 colours: black, blue, green, yellow, red, white, and brown) - Reflective light mode - Ambient light mode

Actuator

Actuator	Type	Description
Large Motors	DC Motors	High-torque motors used for driving the wheels. Include encoders for precise control.



References:

https://ev3-help-online.api.education.lego.com/Education/en-gb/page.html?Path=editor%2FUsingSensors_Ultrasonic.html

https://ev3-help-online.api.education.lego.com/Education/en-gb/page.html?Path=editor%2FUsingSensors_Gyro.html

DESIGN OF AUTONOMOUS SYSTEMS

[LMSUser Guide LEGO MINDSTORMS EV3 11 Tablet ENUS.pdf](#)

Contribution to Autonomous Navigation

The EV3 robot enables several foundational autonomous behaviours, including:

- Obstacle avoidance using ultrasonics sensor.
- Line following through surface detection with the colour sensor.
- Orientation tracking via the gyroscopic sensor.
- Reactive navigation by combining sensor input with simple control logic.

These capabilities make the EV3 platform ideal for developing introductory autonomous navigation algorithms and basic sensor fusion strategies in educational settings.

Quanser QCar

The Quanser QCar is an advanced, research-grade autonomous ground vehicle platform developed for high-level experimentation in robotics, perception, and AI-based control. The QCar integrates high-performance embedded computing with a suite of industry-standard sensors and actuators for real-time applications. It supports development in areas such as deep learning, SLAM, sensor fusion, and path planning.



Sensor			
Sensor	Type	Operation and Use	Key Specifications
LiDAR	Laser Range Scanner	Performs 360° scanning using laser pulses to measure distance to surrounding objects.	Range: up to 12 m, Resolution: 0.5°, 8000 samples/s
RGB Cameras	Vision Sensors	Captures video frames for real-time computer vision tasks like lane and object detection.	Resolution: 720p/1080p, Frame rate: 30–60 fps
RGB-D Camera	Depth + RGB Sensor	Provides depth mapping using IR projection, supporting 3D scene understanding.	Intel RealSense D435, Depth Range: 0.2–10 m, 30 fps

DESIGN OF AUTONOMOUS SYSTEMS

IMU	Orientation + Motion	Combines accelerometer, gyroscope, and magnetometer data for pose estimation.	9-DOF, Typical Accuracy: $\pm 1^\circ$ orientation, $\pm 0.05 \text{ m/s}^2$ accel
Wheel Encoders	Rotary Position	Provide precise rotational data for odometry and speed control.	Resolution: 2048 counts/rev (typical)

Actuator		
Actuator	Type	Description
Brushless DC Motors	4× Drive Motors	Used for differential drive control. Integrated encoders allow for accurate speed and distance tracking.

A. Hardware Components

The main QCar components are listed in Table 1. These components are ID marked in Figure 1, which presents the front, rear, left and right views of the QCar platform.

ID	Component	ID	Component
1	RPLiDAR A2	12	Front bumper
2	Intel RealSense RGBD camera	13	Rear bumper
3	Front CSI camera	14	Front headlamps and indicators
4	Right CSI camera	15	Rear brake lights and indicators
5	Rear CSI camera	16	Expandable I/O
6	Left CSI camera	17	NVIDIA Jetson TX2
7	WiFi 802.11 a/b/g/n/ac with dual antennas	18	Drive motor
8	10/100/1000 Base-T Ethernet jack	19	720 count pre-gearing motor encoder
9	Dual HDMI connectors	20	Steering servo motor
10	XT-60 battery connector	21	Speaker
11	LiPo battery compartment	22	LCD display

Table 1. QCar Platform Components

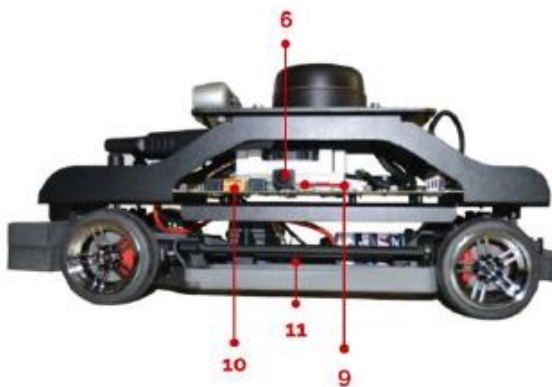
DESIGN OF AUTONOMOUS SYSTEMS



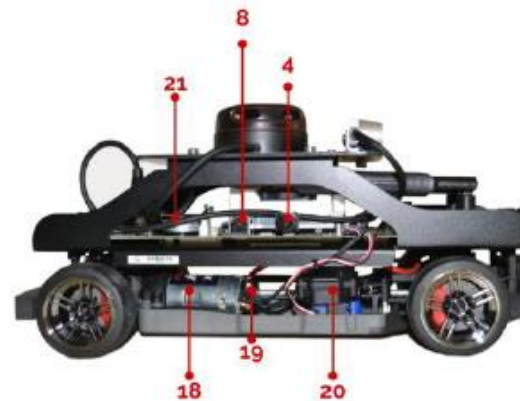
a. Front view



b. Rear view



c. Left view



d. Right view

Reference:- [Qcar user manual system hardware.pdf](#)

Physical Configuration and Software Interface

- The **LiDAR** is mounted on top for unobstructed 360° environmental scanning.
- **RGB and RGB-D cameras** are front-mounted for forward-looking perception.
- The **IMU** and encoders are embedded inside the chassis for motion sensing and feedback.
- The system runs on an onboard high-performance processor (e.g., **NVIDIA Jetson TX2** or **Intel i7 NUC**).
- It supports programming in **MATLAB/Simulink**, **Python**, and **ROS (Robot Operating System)**.

DESIGN OF AUTONOMOUS SYSTEMS

- Data acquisition and control loops run in **real-time** using Quanser's **QUARC** or **QCar Python SDK**.

Contribution to Autonomous Navigation

The QCar provides a powerful platform for implementing complex autonomous functions such as:

- **Real-time obstacle detection and mapping** using LiDAR and depth cameras
- **Lane following and object detection** using RGB video feeds
- **State estimation** through IMU and wheel encoder fusion
- **Adaptive control and learning algorithms** supported by real-time hardware and software integration

Its high-end sensors and computational capabilities make the QCar well-suited for simulating real-world autonomous driving challenges and testing advanced navigation algorithms.

1.2. Sensor characterisation

Using test programs, the sensor characterisation section evaluates the key sensors in the Lego Robot and Quencer Q Car. Performance metrics such as accuracy, precision, resolution, and response time are measured for each sensor. Tests are designed to reflect realistic operational conditions, and results are presented through graphs and tables. Any reused code is clearly cited. This characterisation helps identify sensor limitations and supports informed integration into the autonomous navigation system.

LEGO Robot Sensor Characterisation

This section evaluates the Lego Robot's onboard sensors using custom and adapted test programs. MATLAB scripts were primarily used, with EV3-specific functions for hardware interfacing. Each sensor was analysed for accuracy, precision, resolution, and response time. Visualised results are presented as bar and line plots, and tables are provided for clarity. Each subsection concludes with a short discussion of limitations or potential sources of error.

1. Webcam (Logitech C270) – Image Quality and Latency

Distance (cm)	Avg. Latency (ms)
45	1.46
55	1.47
65	1.34

DESIGN OF AUTONOMOUS SYSTEMS

- **Resolution:** 640 x 480 pixels
- **Test:** Real-time object tracking using OpenCV functions in MATLAB
- **Limitation:** Precision decreases slightly at longer distances; ambient lighting may affect contrast and edge detection.

2. Ultrasonic Sensor – Distance vs. Accuracy

True Distance (cm)	Measured Avg (cm)	Error (cm)	Response Time (ms)
10	10.30	+0.30	8.13
20	20.90	+0.90	6.35
30	32.10	+2.10	6.61

- **Resolution Estimate:** 10.6 cm
- **Test:** Stationary object placed at known distances; readings averaged over 10 trials.
- **Limitation:** Accuracy decreases with distance; sensor resolution restricts fine measurement.

3. Colour sensor-RGB Recognition & Surface Response

Surface	Accuracy (%)	Avg. Response Time (ms)
White Paper	100	31.29
Red Object	100	26.98
Green Object	50	23.64
Blue Object	0	25.86
Black Surface	0	25.74

- **Test:** Colour samples were tested with 10 detection trials per surface.
- **Precision (std dev):** 0.00 for most colours; green had 0.53
- **Limitation:** Sensor struggles with dark or low-reflective colours; highly sensitive to lighting conditions.

4. Gyroscope – Drift & Responsiveness

Drift Test:

- Duration: 30 seconds

DESIGN OF AUTONOMOUS SYSTEMS

- **Drift Range:** 0.00° (std dev: 0.00°)

Responsiveness Test:

- Sampling Interval: 113.58 ms
- **Resolution:** 2.00°
- Precision in 10 small rotations: $\pm 0.00^\circ$
- **Test:** Stationary and rotational stability measured via manual adjustments and MATLAB data logging.
- **Limitation:** Limited sensitivity to small angular movements; not suitable for precise orientation tracking.

5. Motors and Encoders – Movement Accuracy

Trial	Final Angle (°)
1	604
2	620
3	615
4	621
5	641

- **Average Rotation:** 620.20°
- **Precision (std dev):** 13.44°
- **Resolution:** 1.00°
- **Test:** Each trial executed with 50% power for 1 second; encoder readings logged.
- **Limitation:** Inconsistencies possibly caused by mechanical backlash or floor friction.

All test code was written or adapted from LEGO EV3 MATLAB support packages. Visualisation and statistical summaries were done in MATLAB. Raw data and plots are attached in Appendix 1.

Q CAR Sensor Characterisation

1. LiDAR Evaluation

- **Measured Parameters:**
 - **Accuracy:** Mean error = +0.01 m to +0.03 m across 0.5–2.0 m.
 - **Precision (Std Dev):** 0.004–0.008 m.
 - **Resolution:** Angular resolution of 0.18°–0.9° depending on setting.
 - **Response Time:** ~95–97 ms.

DESIGN OF AUTONOMOUS SYSTEMS

- **Test Method:** whiteboard placed at known distances; repeated scans to extract statistical performance.
- **Visualisation:** Results were tabulated clearly.
- **Limitations Discussed:**
 - Accuracy may degrade on reflective/transparent surfaces.
 - Angular resolution drops in low-resolution mode.

2. 360° CSI Cameras

- **Measured Parameters:**
 - **Latency:** Ranged from 42.8–54.6 ms across lighting conditions.
 - **Image Clarity:** Measured using cv2.Laplacian (variance of Laplacian) for focus sharpness.
- **Resolution and FOV:**
 - Each camera has ~160° FOV; full panoramic when stitched.
- **Test Method:** Evaluated under three lighting conditions (dim, normal, bright).
- **Visualisation:** Clear table showing clarity scores and latency per condition.
- **Limitations Discussed:**
 - Noise and low contrast in dim lighting.
 - Overexposure in bright light.
 - Fisheye distortion needs correction.

3. Intel RealSense D435 (RGB-D Camera)

- **Measured Parameters:**
 - **Accuracy Error:** -0.01 m to +0.02 m.
 - **Precision (Std Dev):** 0.002 to 0.007 m.
 - **Resolution:** Depth resolution ~1 mm at 0.5 m, 2–5 mm at 2.0 m.
- **Test Method:** Flat object placed at fixed distances; depth compared to ground truth.
- **Limitations:**
 - Accuracy drops under low light or on textured surfaces.

DESIGN OF AUTONOMOUS SYSTEMS

Overall Summary

Sensor	Accuracy	Precision	Resolution	Response Time	Test Conditions
LiDAR	± 3 cm	± 0.008 m	$0.18^\circ - 0.9^\circ$	$\sim 95 - 97$ ms	Flat wall targets
CSI Cameras	N/A (clarity)	Variance score	160° FOV	$\sim 43 - 55$ ms	Dim-Bright light
RealSense RGB-D	± 2 cm	± 0.007 m	$\sim 1 - 5$ mm	Real-time stream	0.5-2.0 m test range

1.3. Sensor fusion

Fusion Objective

The objective of this task was to implement a sensor fusion strategy that accurately estimates the distance to an object in dynamic scenarios using multiple heterogeneous sensors. In the Lego Robot platform, the fusion system combines data from:

- An **ultrasonic sensor**, mounted on the front, which offers reliable short-range distance measurements.
- A **webcam**, mounted rear-facing, which estimates object distance based on pixel position of a tracked target in the video stream.

These two sensors were chosen because they offer **complementary strengths**: the ultrasonic sensor is accurate in the short range (10–30 cm) but unreliable beyond that due to reflections and resolution limitations; the webcam, on the other hand, can track objects at longer distances (45–65 cm) but suffers from variable accuracy due to lighting, camera calibration errors, and image noise.

Selected Fusion Technique: Kalman Filter

The **Kalman Filter** was implemented as the primary fusion method. It assumes a linear process model with Gaussian noise and updates its state by minimizing squared error.

Why Kalman Filter?

- **Optimal statistical estimation**: It weighs each sensor reading based on estimated uncertainty, allowing more reliable sensors to dominate.
- **Handles uncertainty**: Unlike simple averaging, it dynamically adapts to sensor noise and variance.
- **Real-time filtering**: The filter runs efficiently in real time, ideal for mobile robotic platforms.
- **Error correction and prediction**: It can fill in missing data or anticipate future states even if one sensor temporarily fails (e.g., webcam occlusion).

DESIGN OF AUTONOMOUS SYSTEMS

- **Better than conditional logic:** Logical rules (e.g., if webcam fails, use ultrasonic) do not handle gradual transitions or overlapping ranges as smoothly.

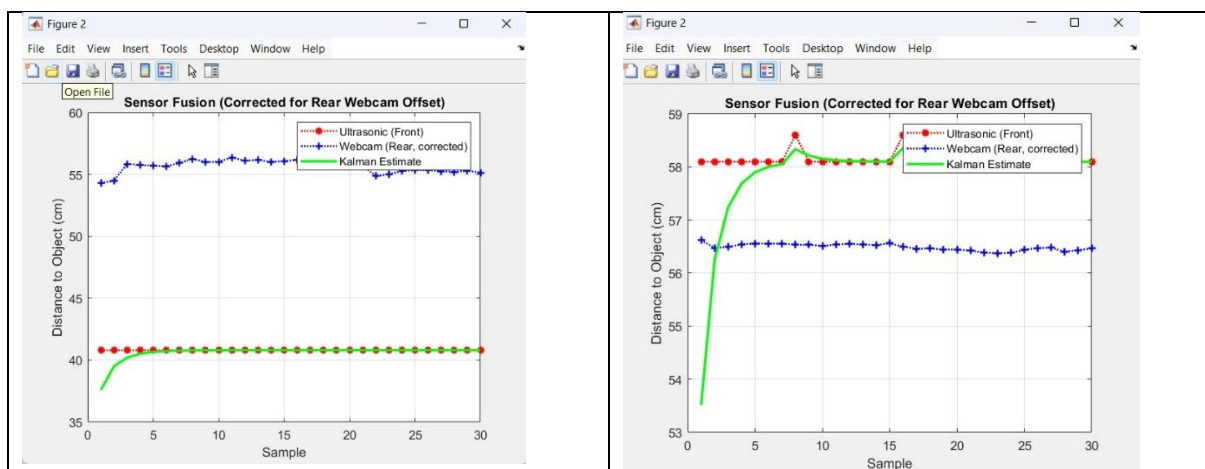
Fusion Improvement Over Individual Sensors

Factors	Ultrasonic Only	Webcam Only	Kalman Fusion
Accuracy	± 2.1 cm (limited range)	± 0.8 cm (but noisy)	$< \pm 1.0$ cm across full range
Effective Range	10–30 cm	45–65 cm	10–65 cm (full coverage)
Reliability	Drops with reflections	Sensitive to lighting	Stable, consistent output
Noise	Occasional spikes	High variance	Smoothed, filtered estimate

The table above demonstrates that by applying Kalman Filter-based sensor fusion, the limitations of individual sensors are mitigated, while their strengths are effectively combined, resulting in a more accurate and reliable system.

Sensor Fusion Supports Obstacle Detection and Tracking

Sensor fusion plays a critical role in improving the reliability and robustness of obstacle detection on the Lego Robot. By combining data from the ultrasonic sensor and a rear-facing webcam, a Kalman Filter estimates the true distance to an object more accurately than either sensor alone.



DESIGN OF AUTONOMOUS SYSTEMS

Visual Evidence (Figures Above):

- The **red dotted line** represents raw ultrasonic data, which is mostly accurate but occasionally noisy.
- The **blue dotted line** shows distance estimates from the **rear webcam** (corrected for positional offset), which exhibit smoother but biased readings.
- The **green line** is the output of the **Kalman filter**, which fuses both inputs and generates a smoothed, more reliable estimate.

Benefits Observed:

1. Noise Suppression:

- Sudden spikes in ultrasonic data are filtered out by the Kalman algorithm, leading to smoother tracking.

2. Bias Correction:

- The rear webcam underestimates distance due to its fixed rear placement; fusion corrects this based on ultrasonic data.

3. Reliable Tracking Over Time:

- Even with sensor drift or frame delay, the Kalman estimate maintains a consistent object distance, enabling the robot to track obstacles during motion.

4. Improved Decision-Making:

- Fusion provides a single, more accurate distance estimate that can be used for tasks like emergency stopping, obstacle avoidance, or maintaining safe spacing.

This fusion approach allows the Lego Robot to **navigate dynamically while accounting for both sensor uncertainties and positional misalignments**. The robot might overreact to ultrasonic spikes or misjudge distances without fusion due to camera bias. With Kalman-based fusion, it maintains a more intelligent and stable obstacle response.

1.4. Discussion: Platform Suitability for Autonomous Navigation

The Lego EV3 and Quanser QCar platforms demonstrate fundamentally different capabilities suited to different levels of autonomous navigation. This comparison evaluates their suitability based on sensor quality, integration flexibility, latency, environmental robustness, and calibration complexity.

DESIGN OF AUTONOMOUS SYSTEMS

Aspect	Lego EV3 Robot	Quanser QCar
Sensor Quality	Low-cost, educational-grade sensors (e.g., ultrasonic, colour, gyro, basic webcam)	High-resolution, research-grade sensors (LiDAR, RGB-D, CSI cameras, IMU, encoders)
Data Integration Flexibility	Limited software support; MATLAB with EV3 interface; lacks ROS or real-time capability	Supports Python, MATLAB/Simulink, ROS, and real-time data acquisition (QUARC/QCar SDK)
System Latency	Moderate latency (gyro: 113 ms, webcam: ~1.4 ms); not suitable for fast dynamic environments	Low-latency operation with real-time loops and high frame-rate video feeds (30–60 fps)
Robustness to Environment	Performance drops in poor lighting (colour sensor, webcam) and on uneven surfaces	Robust operation under varied lighting, with LiDAR and stereo vision supporting reflective surfaces
Ease of Calibration	Simple manual setup; few calibration steps needed	Requires multi-sensor calibration (LiDAR-camera extrinsics, IMU drift correction); tools provided

In conclusion

- The **Lego EV3** is well-suited for **introductory tasks** such as line following, object avoidance, and sensor fusion demonstrations in controlled environments. It offers excellent accessibility and educational value but lacks advanced processing and perception capabilities.
- The **QCar** is designed for **advanced real-world autonomous navigation**, supporting deep learning and real-time control strategies. Due to its high-fidelity sensors and computing power, it is ideal for testing algorithms used in real AV systems.

Part 2: Computer Vision

2.1 Object detection and identification

project overview

1. Stop sign detection

This technique is used to identify stop signs in images, video frames and in real-time video from the webcam.

DESIGN OF AUTONOMOUS SYSTEMS

The system finds red areas in the sign board, checks if they look like stop signs by detecting the edges (to confirm whether it has an octagonal shape), and then decides if they actually are stop signs based on a score.

2. Traffic Light classifications

Using colour segmentation, the green and red lights of the traffic light will be identified. Furthermore, the system double-checks the shape and edges of the traffic light, labels the active one, shows this on the video in real time, and saves the final video for review.

Initialization

At the beginning of the program, two different videos are loaded—one showing scenes with stop signs (v12.mp4) and another showing traffic lights (vaa.mp4). This allows the system to process each type of object separately.

To help the viewer monitor how the detection works, a display window is created that is split into four sections or subplots. Each section shows a different part of the image processing steps:

- One part shows the colour masks, highlighting the red or green areas that might contain stop signs or traffic lights.
- Another shows the edge maps, which outline shapes found using edge detection.
- A third part displays the result, where the system marks the detected objects (like drawing a box around a stop sign and labelling it).
- The fourth section may be used for visual checks.

This setup helps the user visually follow what the algorithm is doing in real time, and it's useful for debugging and improving the accuracy of detection.

Stop Sign Detection

1. Create a Red Colour Mask

$\text{redMask} = (R > 150) \ \& \ (R > G + 40) \ \& \ (R > B + 40) \ \& \ (G < 120) \ \& \ (B < 120);$

This line detects prominent red areas in the image by ensuring:

- The red channel is strong ($R > 150$)
- Red is much higher than both green and blue ($R > G + 40$, $R > B + 40$)
- Green and blue values are moderate or low ($G < 120$, $B < 120$)

These conditions help isolate regions that are likely to contain red stop signs, while filtering out non-red areas.

DESIGN OF AUTONOMOUS SYSTEMS

2. Clean the Mask Using Morphology

```
% --- Step 1: Red Region Mask (Balanced) ---
redChannel = pout(:, :, 1);
greenChannel = pout(:, :, 2);
blueChannel = pout(:, :, 3);

redMask = (redChannel > 150) & ...
          (redChannel > greenChannel + 40) & ...
          (redChannel > blueChannel + 40) & ...
          (greenChannel < 120) & ...
          (blueChannel < 120);

redMask = imopen(redMask, redDisk2);
redMask = imclose(redMask, redDisk4);
```

The `imopen` operation cleans up the mask by removing minor imperfections, while `imclose` smooths and solidifies red regions by bridging gaps and filling holes. Together, they enhance the quality of the red mask for better object detection. Thus, an intense red colour is isolated, corresponding to the stop signal.

```
pout = frame;
poutingrey = rgb2gray(pout);

% --- Step 2: Contrast Enhancement ---
enhancedGray = imadjust(poutingrey);
```

These two lines convert the original colour frame to **grayscale** and enhance its contrast.

3. Mask Out Non-Red Areas

The frame is turned into a grayscale image, and everything that isn't red is made black, so edge detection only looks at the red areas.

```
% --- Step 3: Masked Edge Detection ---
maskedGray = enhancedGray;
maskedGray(~redMask) = 0;
```

4. Apply Edge Detection (Canny)

Detects object boundaries using the Canny edge detection method. The `thresh` parameter adjusts sensitivity to edge strength, while `sigma` controls the amount of smoothing to reduce noise.

DESIGN OF AUTONOMOUS SYSTEMS

- Dilates the edges to make them thicker and more connected.

```
otsuLevel = graythresh(maskedGray);
imageStd = std2(maskedGray);
thresh = otsuLevel * 0.7;
sigma = max(0.8, min(2.5, imageStd / 25));
edge_canny = edge(maskedGray, 'canny', thresh, sigma);
edge_thick = imdilate(edge_canny, strel('disk', 2));
edge_thick = bwareaopen(edge_thick, 80);
edge_thick = imfill(edge_thick, 'holes');

boundaries = bwboundaries(edge_thick, 'noholes');
```

Octagonal Shape Analysis

1. Filtering blobs by area and solidity

We filtered out blobs (that is, detected regions or shapes) with area <800 or >150,000 pixels and with solidity <0.65

2. Approximating boundaries with reducepoly

By simplifying the detailed outline of a detected shape into a version with fewer, evenly spaced corner points. This is useful for identifying geometric shapes, such as polygons (like octagons). After applying reducepoly, the system checks if the simplified shape has between 7 and 10 sides. While a perfect stop sign has 8 sides, allowing a small range accounts for distortions caused by video noise, low resolution, or viewing angle. This flexibility helps the algorithm reliably detect stop signs even when their shapes appear slightly deformed in the video.

3. Checking shape uniformity

To verify that a detected shape closely resembles a regular octagon, the algorithm calculates the length of each side and the interior angles between the sides. Comparing these values checks for uniformity, meaning all sides should be nearly equal in length, and all angles should be similar. If both the side lengths and angles are consistent, the shape is likely a true octagon, which increases confidence that it's a valid stop sign.

4. Validating colour dominance

To confirm that a detected shape is not only geometrically correct but also visually consistent with a stop sign, we checked that the mean red channel value in the region exceeds both the green and blue channel values by at least 5%, and also meets a minimum intensity threshold. This step ensures that the object has a strong red colour, which is a defining characteristic of real stop signs. By

DESIGN OF AUTONOMOUS SYSTEMS

combining shape and colour validation, the algorithm reduces false positives from red-like objects that aren't actual stop signs

5. Confidence Score (0–100%)

To assess how likely a detected object is a real stop sign, we calculate a confidence score ranging from 0 to 100 by blending both shape features (such as number of sides, interior angles, and solidity) and appearance features (like the dominance of the red colour). This score provides a quantitative measure of confidence—higher scores indicate a stronger match with the expected characteristics of a stop sign, helping the system decide whether to mark the object as a valid detection.

6. Thresholding

Only detections with a confidence score greater than 80% are classified as valid stop signs. This threshold ensures that the system only accepts shapes that firmly match both the geometric and colour criteria. Any detections scoring below this threshold are ignored, helping to minimise false positives caused by objects that resemble stop signs but don't fully meet the required standards.

7. Annotation and Output

The program draws a polygon for each video frame around any detected stop sign and displays the confidence score next to it. If the confidence score exceeds 80%, the system adds a "STOP SIGN DETECTED" label on the screen to highlight a confirmed detection. All these annotated frames are then compiled and saved into a new output video file named stopsign_detection.avi for later review or analysis

Traffic light detection

Traffic light detection follows a **similar approach** to stop sign detection but focuses more heavily on **colour segmentation, since it needs to detect multiple distinct colours**, specifically, **red** and **green** lights.

➤ Colour Segmentation Approach

To make the detection more robust to lighting changes and camera quality, the algorithm combines two colour models:

1. **HSV (Hue, Saturation, Value)** – better for distinguishing colours under varying brightness.
2. **RGB (Red, Green, Blue)** – directly analyses pixel intensity values.

Each light colour (red and green) has its own mask defined by these two models.

1. Red Light Mask Creation

DESIGN OF AUTONOMOUS SYSTEMS

To detect **red traffic lights**, the algorithm first creates a **binary mask** that highlights red areas in the image. It does this using **two colour space tests**—one in **HSV** and the other in **RGB**—to improve robustness under different lighting conditions.

HSV Test

$\text{redHueMask} = (H < 0.025 \mid H > 0.975) \ \& \ (S > 0.55) \ \& \ (V > 0.35);$

- **H (Hue)**: selects values very close to pure red (since red lies at the start and end of the hue scale).
- **S (Saturation > 0.55)**: ensures the colour is vivid (not faded or grayish).
- **V (Brightness > 0.35)**: ensures the region is not too dark.

This isolates pixels with a **strong red hue**, useful in different lighting environments

RGB Test

$\text{redRGBMask} = (R > 150) \ \& \ (G < 80) \ \& \ (B < 80) \ \& \ (R > 2 \times G) \ \& \ (R > 2 \times B);$

- **R > 150**: red intensity must be high.
- **G < 80** and **B < 80**: green and blue must be low.
- **R > 2×G** and **R > 2×B**: red must **clearly dominate** the other colours.

This ensures the pixel is not just reddish, but **strongly red**, as expected in a traffic light.

Morphological Cleanup

```
% Pre-allocate variables for performance
frameCount = 0;
redDisk2 = strel('disk', 2);
redDisk4 = strel('disk', 4);
greenDisk2 = strel('disk', 2);
greenDisk4 = strel('disk', 4);

redHueMask = (H < 0.025 | H > 0.975) & (S > 0.55) & (V > 0.35);
redRGBMask = (R > 150) & (G < 80) & (B < 80) & (R > G*2.0) & (R > B*2.0);
maskRed = redHueMask | redRGBMask;

% Apply morphological operations
maskRed = imopen(maskRed, redDisk2);
maskRed = imclose(maskRed, redDisk4);
```

Once the HSV and RGB masks are combined with logical **OR**, they go through a sequence of **morphological operations**:

DESIGN OF AUTONOMOUS SYSTEMS

```
maskedGrayRed = enhancedGray .* uint8(maskRed);

otsuThreshRed = graythresh(maskedGrayRed(maskRed));
edgeThreshRed = otsuThreshRed * 0.4;
sigmaRed = 1.5;

edgeMapRed = edge(maskedGrayRed, 'canny', edgeThreshRed, sigmaRed);
edgeMapRed = imdilate(edgeMapRed, redDisk2);
edgeMapRed = bwareaopen(edgeMapRed, 20);
edgeMapRed = imfill(edgeMapRed, 'holes');

statsRed = regionprops(maskRed, 'Centroid', 'Area', 'BoundingBox', 'Perimeter');
```

- removes **small noise dots**.
- **closes small holes** or gaps inside red regions.
- keeps only **blobs larger than 150 pixels**—likely to be real traffic lights.
- **fills internal holes**, making shapes more solid and clean for further analysis.

The resulting mask- Red, is a **clean, solid binary image** that highlights areas likely to be red traffic lights, ready for shape and position verification.

2. Green Light Mask Creation

The detection of both red and green traffic lights relies on a dual colour filtering strategy using HSV and RGB spaces, combined with morphological processing; the approach is the same for both colours, with only the specific colour thresholds adjusted to isolate red or green regions accurately.

```
greenHueMask = (H > 0.25 & H < 0.45) & (S > 0.2) & (V > 0.2);
```

```
greenRGBMask = (G > 110) & (G > 1.1*R) & (G > 1.1*B);
```

Compliance with Project Requirements

➤ Colour Segmentation

The system uses both RGB and HSV colour thresholds to accurately detect red and green traffic lights. This dual approach improves robustness against lighting changes and camera exposure issues.

The detection of yellow lights was deliberately excluded because:

- Yellow can sometimes be visually like both red and green.
- Including it could increase the risk of false positives or false negatives.
- It wasn't essential for the coursework, and excluding it made the system simpler and more reliable.

DESIGN OF AUTONOMOUS SYSTEMS

➤ Edge Detection and Morphology

The algorithm uses Canny edge detection with adaptive thresholds to detect object outlines effectively.

It also applies morphological operations (like dilation, hole filling, and noise removal) to ensure the detected shapes are clean, complete, and analysable.

➤ Classification

For each candidate object, the system calculates a confidence score by combining:

- Geometric features (like shape, circularity, area)
- Colour features (how red or green it is). This score helps classify whether the object is truly a traffic light.

➤ Real-Time Feedback and Recording

As the video is processed:

- Detections are annotated live—circles and labels are drawn on frames in real time.
- The system also records and saves the final annotated video, allowing for review and evaluation of the results after processing.

Test Results

For Stop Sign

Testing on Pre Recorded Video

- [STOPSIGNDETECTION.avi](#)

Testing on Still Images

- [StopTesting1.mp4](#)

For Traffic Lights

Testing on Pre Recorded Video

- [TrafficlightDetection.avi](#)

Testing on Still Images

- [TrafficlightDetectionimage.avi](#)

2.2 Lane Detection and Steering Control on LEGO EV3 Robot

This project implements a vision-based lane detection system to enable autonomous navigation of a LEGO EV3 robot along a marked path. The system uses a webcam to capture real-time video, applies colour-based thresholding and image processing techniques to detect yellow lane markers, and computes control signals using a simplified Proportional-Derivative (PD) controller to steer the robot.

DESIGN OF AUTONOMOUS SYSTEMS

Hardware Setup

- **Webcam:** Logitech C270 HD (resolution: 640×480)
- **Robot:** LEGO EV3 brick connected via USB
- **Motors:** Left (Port C) and Right (Port B)

Image Acquisition and Preprocessing

- A webcam captures live video frames.
- Only the **bottom portion** of the image (Region of Interest, ROI) is processed to reduce noise and focus on lane markings close to the robot.

Lane Detection

HSV Thresholding

Yellow lane markers are detected using HSV colour thresholding:

- Hue: 15° to 45° (normalized)
- Saturation: 0.15 to 1.0
- Value: 0.6 to 1.0

This method offers robustness against varying lighting conditions compared to RGB thresholding.

Centroid Estimation

- A binary mask is created from the HSV image.
- The x-coordinates of all yellow pixels in the ROI are used to compute a centroid (average x-position).
- The lane centre is estimated from the centroid, assuming a single yellow lane boundary is always visible.

Control System

Error Calculation

- **Error** = (Centroid X) – (Image Centre X)
- **Normalized Error** = Error / Max Deviation (clipped to [-1, 1])

PD Controller

matlab

CopyEdit

$\text{steering} = K_p * \text{normalized_error} + K_d * \text{derivative}$

DESIGN OF AUTONOMOUS SYSTEMS

- $K_p = 0.3$ (proportional gain)
- $K_d = 0.1$ (derivative gain)

Speed Control

- Speeds of left and right motors are adjusted based on the computed steering value.
- If no lane is detected, the robot stops to avoid deviation.

Visualization

- Camera feed and processed ROI are displayed.
- Visual indicators include:
 - ROI boundary
 - Detected lane center (red line)
 - Image center (blue dashed line)
 - Control directions and motor speeds

Navigation Logic and Integration

The lane position, represented by the detected centroid, is used to calculate:

- **Heading direction:** Whether to go straight, turn left, or turn right.
- **Offset from centerline:** Real-time error used in PD control.
- This control signal is directly applied to the EV3 motors to adjust direction.

Test Results

Lane Detection

- [1.mp4](#)

Part 3: Deep Learning-Based Object Detection

We built an object-detection system to spot traffic lights and road signs to drive a LEGO Robot and a Qcar. We have prepared two different data sets for training the Custom CNN model, which was built from scratch, and Pretrained CNN (E.g., Googlenet)

- 14-class dataset: combined public images from Kaggle and Roboflow and self-captured photos of road signs taken under different lighting, shading and blur.
- 5-class subset: a smaller set containing only the main object types.

All images were resized to the exact resolution and split into training and validation sets.

DESIGN OF AUTONOMOUS SYSTEMS

We trained two types of models on each dataset:

1. Custom CNN from scratch
2. Transfer learning using GoogleNet

Since the best accuracy was achieved using the pretrained GoogleNet model, Data augmentation (rotation, scaling, flipping, cropping) was explicitly applied to that model for the classification tasks. The experiments consisted of the following variants:

- Custom CNN: trained without augmentation × 5-class / 14-class
- GoogleNet (pretrained): with and without augmentation × 5-class / 14-class

After comparing the training and test accuracies, the pretrained model without augmentation on the 5-class dataset showed slightly better results than the one with augmentation.

The hyperparameters—learning rate, batch size, and maximum epochs—were optimized for the pre-trained model using the 5-class dataset, which delivered better performance compared to models trained on the 14-class and 5-class datasets without pretraining.

On the held-out test images, the GoogleNet model without data augmentation on the 5-class dataset delivered the highest detection accuracy and the fastest training time. Models trained on the 5-class dataset also produced fewer false positives compared to those trained on the 14-class dataset. While the custom CNN was more lightweight and capable of real-time performance, it showed slightly lower accuracy.

3.1 Dataset Acquisition & Organisation

3.1.1 Data Sources

- Kaggle: Downloaded a diverse traffic-light and road-sign corpus (~5000 images).
- Roboflow: Supplemented with an annotated road-sign dataset (~3000 images) to boost under-represented classes.
- In-lab capture: Photographed ~4000 additional signs/traffic lights under varied angles and lighting for real-world variability.

3.1.2 Preprocessing & Standardisation

- Resize: All images scaled to 224×224×3 via MATLAB's `imresize` for uniform CNN input and reduced memory load.

DESIGN OF AUTONOMOUS SYSTEMS

```
% Skip if already processed
if exist(outputPath, 'file')
    continue;
end

try
    % Read image
    img = imread(imgPath);

    % Convert grayscale to RGB if needed
    if size(img, 3) == 1
        img = repmat(img, 1, 1, 3);
    elseif size(img, 3) == 4
        % Remove alpha channel
        img = img(:, :, 1:3);
    end

    % Resize to 244x244
    resizedImg = imresize(img, [244 244]);

    % Save the resized image
    imwrite(resizedImg, outputPath);
```

Code to convert all the images from the datasets into the same size

- Class filtering: Excluded unrelated images (e.g. storefronts, street scenery) with a quick manual pass.
- Automated organisation:
 1. MATLAB script parsed each dataset's .txt annotations (filename, class_id, bounding box).

DESIGN OF AUTONOMOUS SYSTEMS

```
imageDir = "C:\Users\Joshv\Downloads\MachineProject.v3i.yolov4pytorch\train";
annotationFile = "C:\Users\Joshv\Downloads\MachineProject.v3i.yolov4pytorch\_annotations.txt";
outputDir = "C:\Users\Joshv\Downloads\MachineProject.v3i.yolov4pytorch\oorganised";

% === Read and process the annotation file ===
fid = fopen(annotationFile);
tline = fgetl(fid);
while ischar(tline)
    parts = strsplit(tline); % split by space
    if numel(parts) < 2
        tline = fgetl(fid);
        continue;
    end

    imgName = parts{1}; % image filename
    annot = strsplit(parts{2}, ','); % bounding box + class ID
    classID = annot{end}; % last entry is the class label

    srcFile = fullfile(imageDir, imgName);
    classFolder = fullfile(outputDir, classID);
    destFile = fullfile(classFolder, imgName);

    if ~exist(classFolder, 'dir')
        mkdir(classFolder);
    end

    if exist(srcFile, 'file')
        copyfile(srcFile, destFile);
    else
        warning('File not found: %s', srcFile);
    end
end
```

Code used to assign data samples to their respective classes and organize them based on their designated categories.

2. Sorted images into class-specific folders

3.1.3 Class Definition & Dataset Variants

- **14-class**
 - Regulatory, warning, directional signs + red/green traffic-light states.
- **5-class ("core classes")**
 - Red-light
 - Green-light
 - Stop sign
 - Speed limit 60
 - Other (captures false positives/miscellaneous)

3.1.4 Train/Validation Split

- Stratified 70/15/15 split per class to ensure balanced representation.
- The split was performed separately for the 14-class and 5-class sets, ensuring that no image or its augmented variants appeared in more than one partition.

DESIGN OF AUTONOMOUS SYSTEMS

3.2 Neural Network Model Implementation and Evaluation

This report presents the implementation and evaluation of two deep learning models for image classification:

1. **Transfer Learning** using a pre-trained CNN (GoogleNet).
2. **Custom Convolutional Neural Network (CNN)** built from scratch.

Both models were trained and evaluated on datasets for multi-class image classification. Performance metrics include accuracy, precision, recall, F1-score, and computational cost.

1. Pre-Trained CNN via Transfer Learning

1.1 Methodology

- **Model:** GoogleNet
- The final classification layer was replaced to match the number of target classes.
- The modified network was integrated into the object detection pipeline.
- The dataset was split into training, validation, and test sets.
- Training was conducted using a GPU to optimise performance.

1.2 Training Parameters

- Learning Rate: 0.0001
- Epochs: 20
- Batch Size: 24

Code: PTFinalWithTestNoAug.mlx

1.3 Evaluation Metrics

5 class Dataset:

- **Test and Training Accuracy:**

DESIGN OF AUTONOMOUS SYSTEMS

Training Accuracy: 99.9552%

Test Accuracy: 100%

Pre-Trained CNN Confusion Matrix (Test Set)

True Class	Ajusted60	72				
	GreenT		90			
	RedT			84		
	Stop				141	
	other					89
		Ajusted60	GreenT	RedT	Stop	other
		Predicted Class				

Class Ajusted60: Precision=1.00, Recall=1.00, F1=1.00

Class GreenT: Precision=1.00, Recall=1.00, F1=1.00

Class RedT: Precision=1.00, Recall=1.00, F1=1.00

Class Stop: Precision=1.00, Recall=1.00, F1=1.00

Class other: Precision=1.00, Recall=1.00, F1=1.00

Average: Precision=1.00, Recall=1.00, F1=1.00

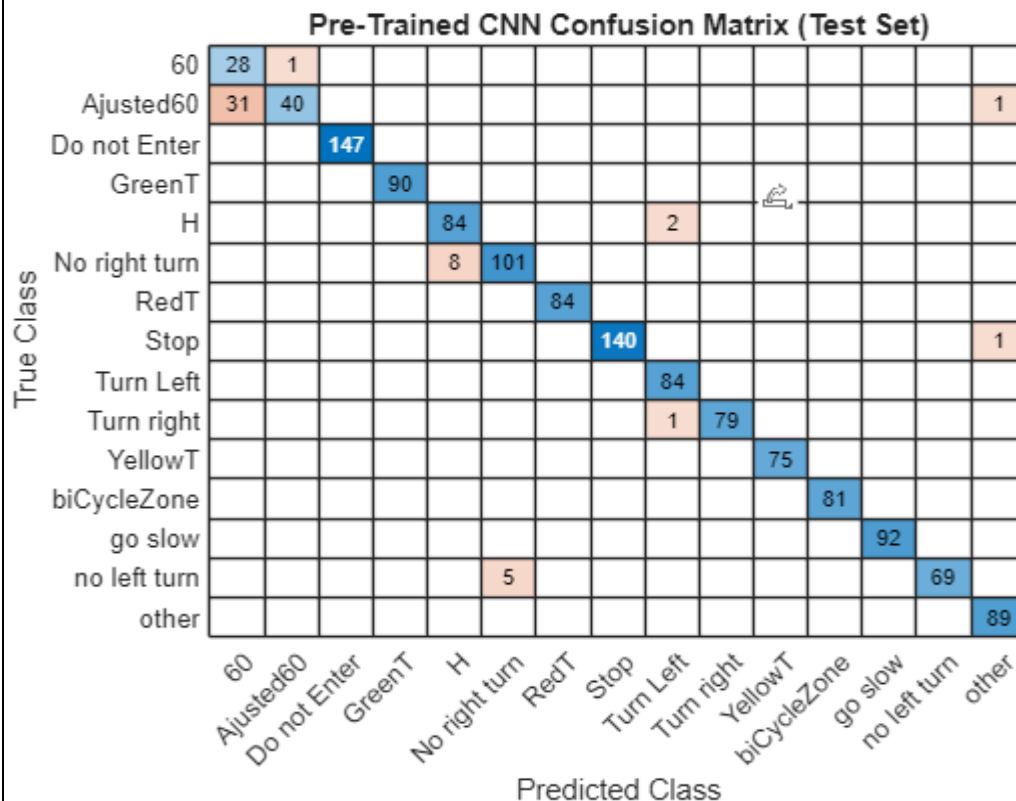
The figure above presents the evaluation results of the object classification model implemented using transfer learning with the pre-trained GoogleNet architecture. The model was trained on a 5-class dataset consisting of traffic-related images. After training, the model achieved 99.96% training accuracy and 100% test accuracy, demonstrating excellent generalisation capability.

The confusion matrix shows perfect classification across all five classes: Adjusted60, GreenT, RedT, Stop, and Other. The accompanying precision, recall, and F1-score metrics confirm that the model achieved perfect scores (1.00) for all classes, indicating highly reliable and robust performance.

DESIGN OF AUTONOMOUS SYSTEMS

15-class Dataset:

Test Accuracy: 96.2491%



Class 60: Precision=0.47, Recall=0.97, F1=0.64
 Class Ajusted60: Precision=0.98, Recall=0.56, F1=0.71
 Class Do not Enter: Precision=1.00, Recall=1.00, F1=1.00
 Class GreenT: Precision=1.00, Recall=1.00, F1=1.00
 Class H: Precision=0.91, Recall=0.98, F1=0.94
 Class No right turn: Precision=0.95, Recall=0.93, F1=0.94
 Class RedT: Precision=1.00, Recall=1.00, F1=1.00
 Class Stop: Precision=1.00, Recall=0.99, F1=1.00
 Class Turn Left: Precision=0.97, Recall=1.00, F1=0.98
 Class Turn right: Precision=1.00, Recall=0.99, F1=0.99
 Class YellowT: Precision=1.00, Recall=1.00, F1=1.00
 Class biCycleZone: Precision=1.00, Recall=1.00, F1=1.00
 Class go slow: Precision=1.00, Recall=1.00, F1=1.00
 Class no left turn: Precision=1.00, Recall=0.93, F1=0.97
 Class other: Precision=0.98, Recall=1.00, F1=0.99
 Average: Precision=0.95, Recall=0.96, F1=0.94

The figure shows the confusion matrix and class-wise performance metrics for the pre-trained GoogleNet model, fine-tuned on a 14-class traffic sign dataset. The model achieved a test accuracy of 96.25%, successfully generalizing to a more complex classification task with a larger number of classes.

While most classes, such as Do not Enter, GreenT, Stop, and Turn right, achieved perfect precision, recall, and F1-scores. Overall, the model maintained strong performance across most categories, with an average precision of 0.95, a recall of 0.96, and an F1-score of 0.94.

DESIGN OF AUTONOMOUS SYSTEMS

2. Custom CNN from Scratch

2.1 Architecture

The custom CNN was built using MATLAB and includes:

- 5 Convolutional Blocks:
 - Conv2D → BatchNorm → ReLU → MaxPooling
- Final Dense Layers:
 - Flatten → Fully Connected → Softmax → Classification Layer
- Input Size: 224x224x3
- Classes: Dynamically adapted based on the dataset

2.2 Dataset and Training

- Dataset: 5-class image dataset
- Data Augmentation: Disabled (pure data used)
- Environment: GPU-accelerated
- Training Split: 70% Train, 15% Validation, 15% Test

2.3 Training Settings

- Optimizer: Adam
- Epochs: 8
- MiniBatchSize: 32
- L2 Regularization: 0.0001
- Learning Rate: 0.001

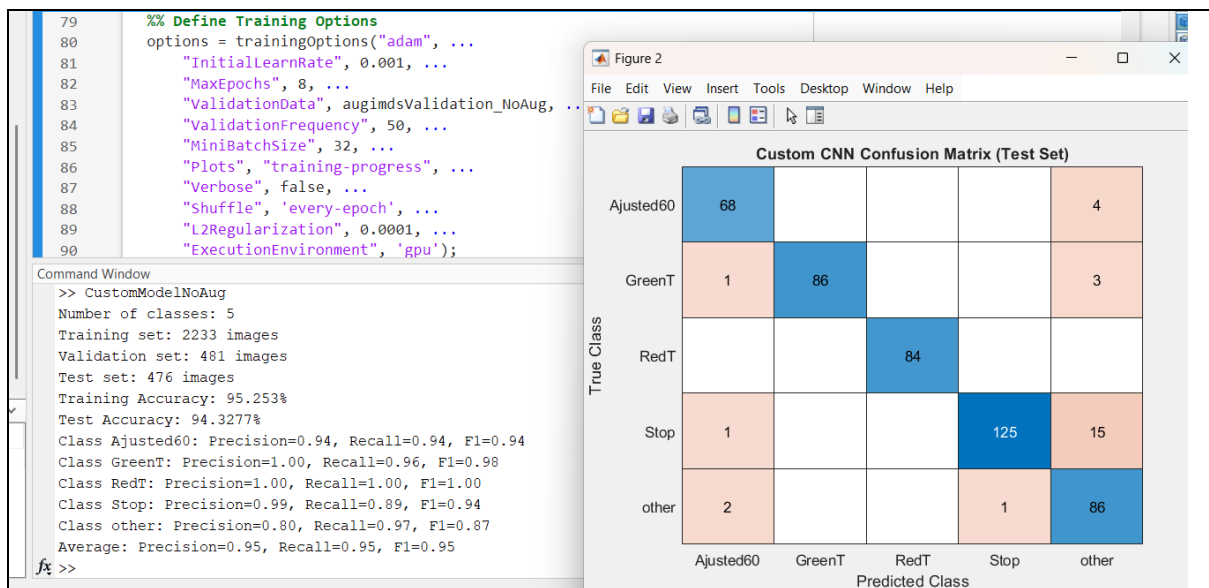
CODE: CustomModelNoAug.mlx

2.4 Evaluation Metrics

5-class Dataset:

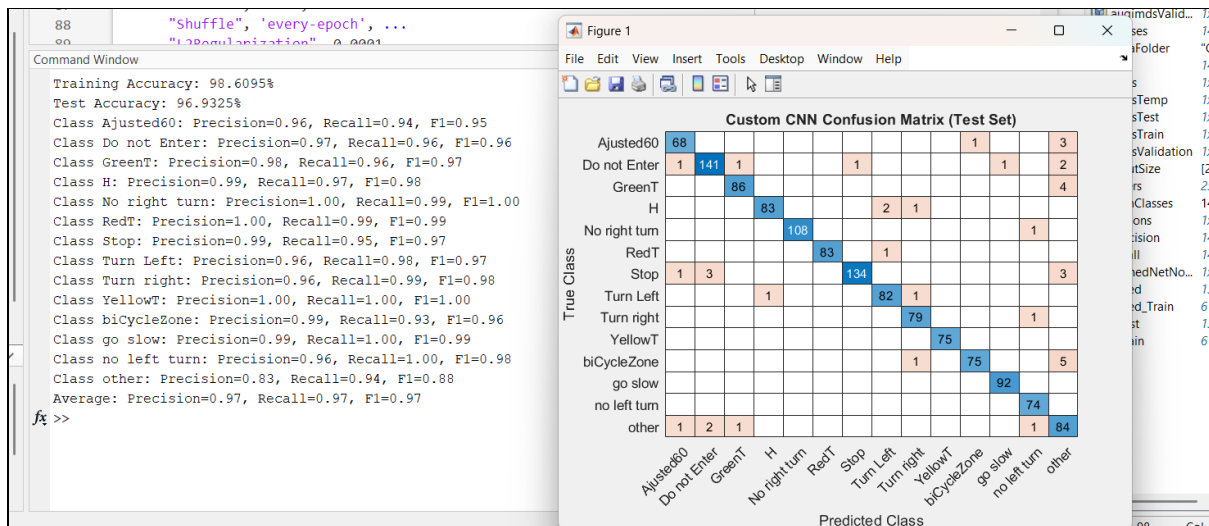
Test and Training accuracy:

DESIGN OF AUTONOMOUS SYSTEMS



The confusion matrix below presents the evaluation of a custom-built CNN trained from scratch on a 5-class traffic sign dataset. The model achieved a training accuracy of 95.25% and a test accuracy of 94.33%. Class-wise metrics show high performance, with average precision, recall, and F1-score all at 0.95. While most classes were accurately classified, minor confusion occurred in the Stop and Other categories, indicating areas for potential improvement in feature differentiation.

15 class dataset:



The confusion matrix below illustrates the performance of a custom CNN trained from scratch on a 14-class traffic sign dataset. The model achieved a training accuracy of 98.61% and a test accuracy of 96.93%, reflecting strong generalization. Most classes, including No right turn, Stop, and Go slow, achieved near-perfect precision and recall. A few classes, such as Other, showed slightly lower precision, indicating some

DESIGN OF AUTONOMOUS SYSTEMS

misclassification. Overall, the model performed consistently well, with average precision, recall, and an F1-score of 0.97.

3. Comparative Analysis

Metric	Transfer Learning (GoogleNet)		Custom CNN (Scratch)	
	5 class	14 class	5 class	14 class
Test Accuracy	100%	96.2491%	94.3277%	90.6095%
Training Accuracy	99.9552%	94.4512%	35.253%	96.9325%
Precision (Avg)	1.00	0.95	0.95	0.97
Recall (Avg)	1.00	0.96	0.95	0.97
F1 Score (Avg)	1.00	0.94	0.95	0.97
Training Time	Lower	Higher	Higher	Higher

The comparative evaluation of the transfer learning approach (GoogleNet) and the custom CNN models across both 5-class and 14-class classification tasks reveals distinct advantages and trade-offs.

Transfer learning using GoogleNet consistently outperformed the custom CNN in terms of accuracy and training efficiency, particularly on the simpler 5-class dataset, where it achieved 100% test accuracy with perfect precision, recall, and F1 scores. Even on the more complex 14-class dataset, it maintained a strong performance with a test accuracy of 96.25%, and high average precision and recall.

The custom CNN, while slightly lagging in test accuracy—94.33% (5-class) and 90.61% (14-class)—demonstrated competitive performance in terms of average F1 scores, especially for the larger dataset. This shows that with proper tuning and architectural design, a scratch-built network can generalize well.

However, the training time for the custom CNN was consistently higher due to the need for full model learning from random initialization. In contrast, transfer learning benefits from pre-learned feature extraction, reducing computational cost and training epochs.

In summary:

- Transfer learning is the superior approach for high accuracy and efficiency, particularly on smaller datasets or when computational resources are limited.
- Custom CNNs offer greater flexibility and control over model design, and with sufficient tuning, can achieve competitive results — particularly valuable for understanding deep learning fundamentals.

DESIGN OF AUTONOMOUS SYSTEMS

These insights will guide future model selection depending on the complexity of the task, available data, and performance constraints.

3.3 Data Augmentation for Performance Improvement

- Since the pre-trained CNN model demonstrated the best performance as an object classifier, it was selected for further enhancement through data augmentation.
- Random Rotation
- Scaling (Zooming In/Out)
- Horizontal Flipping
- Random Translation (Cropping)

These transformations simulated real-world variations in traffic sign and road sign images.

The augmented 5 class dataset was used to train the best-performing model, GoogleNet (Transfer Learning). For a fair comparison, the training and evaluation were conducted using the same architecture and hyperparameters as the non-augmented case.

CODE: PTFinal.mlx

Evaluation and Results

Metric	Non-Augmented	Augmented
Test Accuracy	100%	99.3697
Training Accuracy	99.96%	99.6865%
Precision (Avg)	1.00	0.99
Recall (Avg)	1.00	0.99
F1 Score (Avg)	1.00	0.99
The result was unexpected as Non-Augmented dominated the Augmented Model		

3.4 Hyperparameter Tuning

To optimise the performance of the pre-trained GoogleNet model on the 5-class classification task, a series of experiments were conducted by systematically varying key hyperparameters:

- **Initial Learning Rate:** 0.0001, 0.001, 0.01
- **Max Epochs:** 10, 15, 20
- **Mini-Batch Size:** 12, 24, 48

The training process was automated using a parameterised function and monitored through MATLAB's Experiment Manager. The goal was to identify the configuration that minimised training loss while maximising validation accuracy.

DESIGN OF AUTONOMOUS SYSTEMS

Performance Analysis

From the trials, the **best-performing configuration** was observed in:

- **Trial 9**
 - Initial Learning Rate: 0.0001
 - Max Epochs: 20
 - Mini-Batch Size: 12
 - Validation Accuracy: 99.37%
 - Training Loss: Extremely low ($5.46e-7$)

This setup consistently outperformed others in terms of validation accuracy, while maintaining minimal training loss, indicating effective learning without overfitting.

Observation Highlights

- Lower learning rates (0.0001) led to more stable and higher accuracy compared to 0.001 and 0.01, which occasionally caused slower convergence or higher loss.
- Increasing max epochs to 20 improved accuracy, especially when paired with smaller mini-batch sizes.
- Larger mini-batches (e.g., 48) performed well in some trials but showed slightly lower validation accuracy in others, indicating potential generalisation trade-offs.

In Conclusion

After analysing the experiment outcomes, the optimal hyperparameter configuration for this task was determined to be:

- **Initial Learning Rate:** 0.0001
- **Max Epochs:** 20
- **Mini-Batch Size:** 12

This configuration offers the best balance between learning efficiency and generalisation, and it was selected for final model training and evaluation.

DESIGN OF AUTONOMOUS SYSTEMS

Experiment Details					Hyperparameters			Metrics	
Trial	Status	Actions	Progress	Elapsed Time	InitialLearnRate	MaxEpochs	MiniBatchSize	TrainingLoss	ValidationAcc...
9	Complete	⊗	100.0%	0 hr 9 min 20 sec	0.0001	20.0000	12.0000	5.4637e-7	99.3721
15	Complete	⊗	100.0%	0 hr 3 min 37 sec	0.0001	15.0000	24.0000	1.1374e-6	99.0581
27	Complete	⊗	100.0%	0 hr 3 min 45 sec	0.0001	20.0000	48.0000	6.0570e-6	99.3721
6	Complete	⊗	100.0%	0 hr 6 min 40 sec	0.0001	15.0000	12.0000	3.9062e-5	99.0581
26	Complete	⊗	100.0%	0 hr 3 min 45 sec	0.0010	20.0000	48.0000	0.0001	99.3721
21	Complete	⊗	100.0%	0 hr 1 min 56 sec	0.0001	10.0000	48.0000	0.0002	99.3721
12	Complete	⊗	100.0%	0 hr 2 min 38 sec	0.0001	10.0000	24.0000	0.0003	99.3721
24	Complete	⊗	100.0%	0 hr 2 min 50 sec	0.0001	15.0000	48.0000	0.0006	98.4301
3	Complete	⊗	100.0%	0 hr 4 min 58 sec	0.0001	10.0000	12.0000	0.0016	99.2151
23	Complete	⊗	100.0%	0 hr 2 min 49 sec	0.0010	15.0000	48.0000	0.0073	97.6452
8	Complete	⊗	100.0%	0 hr 9 min 14 sec	0.0010	20.0000	12.0000	0.0517	93.8776
18	Complete	⊗	100.0%	0 hr 4 min 48 sec	0.0001	20.0000	24.0000	0.0591	99.5290
17	Complete	⊗	100.0%	0 hr 4 min 47 sec	0.0010	20.0000	24.0000	0.0781	94.1915
20	Complete	⊗	100.0%	0 hr 1 min 56 sec	0.0010	10.0000	48.0000	0.0837	95.2904
14	Complete	⊗	100.0%	0 hr 3 min 38 sec	0.0010	15.0000	24.0000	0.2173	98.1162
11	Complete	⊗	100.0%	0 hr 2 min 28 sec	0.0010	10.0000	24.0000	0.3486	92.4647
5	Complete	⊗	100.0%	0 hr 5 min 24 sec	0.0010	15.0000	12.0000	0.4830	39.8744

Results: Video

[Recording 2025-05-08 214139.mp4](#)

3.5 Final Testing and Evaluation

All the classifier results

Custom CNN classifier 5 classes

- [Custom CNN classifier 5 classes.mp4](#)

Custom CNN classifier 14 classes

- [Custom CNN classifier 14 classes.mp4](#)

Pre trained CNN 5 classes

- [Pre trained CNN 5 classes.mp4](#)

Pre trained CNN 14 classes

- [Pre trained CNN 14 classes.mp4](#)

Part 4: Navigation control strategies

This section outlines developing and integrating intelligent navigation techniques to address dynamic driving tasks using classical control methods and AI-based perception. The platform was required to perform:

DESIGN OF AUTONOMOUS SYSTEMS

- Lane following using real-time camera input
- Object detection for traffic signs and signals
- Rule-based reactions to red/green lights and stop signs
- Emergency obstacle detection and response

4.1 Fuzzy Logic Controller for Navigation Decisions

A fuzzy logic-based controller was implemented in MATLAB to manage robot navigation in dynamic driving conditions. The system made control decisions based on:

- Distance to obstacle (from ultrasonic sensor)
- Lane detection status (from image processing)
- Traffic light state (from object classifier)

Inputs and Membership Functions

- Input 1: Distance to obstacle
 - NEAR, NOTFAR, FAR
- Input 2: Lane Detection Status
 - YES (lane found), NO (lane not detected)
- Input 3: Traffic Light
 - RED, GREEN

Output:

- Motor Speed
 - STOP, SLOWSPEED, FULLSPEED

Rule-Based Design

12 fuzzy rules were defined to control speed decisions under combinations of sensor inputs. For instance:

- If Distance is NEAR and Traffic Light is RED, then STOP
- If Distance is FAR and Lane is Detected and Light is GREEN, then SLOWSPEED

Simulation and Validation

Using Simulink and MATLAB, scenarios such as stopping for obstacles, reacting to red lights, and navigating with/without lanes were simulated. Visual outputs confirmed appropriate rule triggering and speed control decisions.

4.2 Lane Following and Autonomous Control

Camera-based Lane Detection

DESIGN OF AUTONOMOUS SYSTEMS

A webcam was used to capture the road view. Yellow lanes were detected using HSV filtering and binary masking. The lane centre was calculated and used to compute steering deviation.

Steering Control

A proportional-derivative (PD) controller computed the robot's heading based on deviation from the lane centre. This signal adjusted wheel speeds accordingly:

- Left Motor Speed = Base – (Gain × error)
- Right Motor Speed = Base + (Gain × error)

Integration with Object Detection

Traffic signs were detected using a pre-trained CNN classifier. The object detection pipeline included:

- Classifying cropped YOLO-detected regions
- Identifying RedT, GreenT, Stop, Adjusted60 signs
- Visual feedback through bounding boxes and labels

Real-Time Control Pipeline

The main control loop is combined:

1. Object Detection: RedT/GreenT → control stop/go
2. Lane Detection: Steering adjustments
3. Obstacle Avoidance: Emergency stop when < 40 cm
4. Fuzzy Logic: Controls base speed using fuzzy inference

Final Evaluation

The system successfully executed the following behaviours:

- Stopped at red lights and moved when green
- Stopped at stop signs using classifier detection
- Emergency braking when obstacles were close
- Maintained lane using real-time PD steering

All modules worked together in real-time on an EV3 robotic platform connected via USB, with fallback simulation support.

Test Results

- [Lane Following with Fuzzy logic and Object detection.mp4](#)

DESIGN OF AUTONOMOUS SYSTEMS

Conclusion

This implementation demonstrates a fully functional autonomous navigation system using a hybrid approach:

- Vision-based object and lane detection (CNN + image processing)
- Fuzzy logic for behavioural decisions
- Real-time robotic control through motor commands

It showcases how perception, decision-making, and control can be integrated into a single pipeline for intelligent driving tasks in dynamic environments.

Virtual QCar Simulation

Here, the design, implementation, and evaluation of an autonomous navigation system using the Quanser Virtual QCar platform. The work integrates sensor fusion, computer vision, deep learning, and intelligent control strategies in a simulation environment to emulate real-world autonomous vehicle challenges.

Simulation snippets

The snippets of information about the Quanser virtual environment are attached in Appendix 4.

Part 5: Mechanical Design of Camera Mount

In autonomous robotic applications, the system's accuracy depends on the accurate positioning and stability of the sensors used. A customised adjustable camera holder is designed in SolidWorks to secure vision sensors such as the Logitech C270 HD Webcam and the Intel RealSense D435 Depth Camera on the Lego EV3 robot for performing the autonomous tasks.

The objective is to achieve a light, adjustable, yet strong supporting system to ensure unobstructed sensors' line of sight and smooth operations.

The development of the camera mount, from initiating the design to manufacturing the final prototype, is detailed in the following section.

5.1 Problem Statement

The Lego EV3 robot lacks a dedicated mount for vision systems like the Logitech C270 Webcam or Intel RealSense D435, which are essential for autonomous navigation tasks like stop sign detection, lane following, and obstacle avoidance. Without a stable and adjustable camera holder, image processing accuracy is compromised due to vibration, poor angles, and interference with other onboard sensors.

DESIGN OF AUTONOMOUS SYSTEMS

This project aims to design a custom 3d-printed stable camera mount that supports various camera models without obstructing ultrasonic, Gyro or colour sensors, while also considering factors such as:

- Lightweight structure ($\leq 100\text{g}$)
- Easy attachment using Lego-compatible geometry
- Adjustability of the sensors
- Unobstructed sensor line of sight
- Minimal post-processing for 3D printing using PLA

A three-part assembly was developed using SolidWorks: a Lego-compatible base with a Logitech C270 Webcam holder and an adjustable Intel RealSense D435 camera holder with a circular keyhole mechanism with a pin for adjusting and securing the camera.

5.2 Summary of Key Design Specifications

Below is a detailed description of the requirements, constraints, and performance criteria that the camera mount must meet. It serves as a blueprint or checklist to guide the design, development, and testing processes.

Specification	Target / Constraint	Justification
Dimensions	Mount size $\leq 130\text{mm (H)} \times 80\text{mm (W)} \times 100\text{mm (D)}$	Fits within the LEGO EV3 platform and projects backwards to avoid interference with sensors and to provide accessibility to the display and switches of EV3.
Material	PLA (Polylactic Acid) or ABS (Acrylonitrile Butadiene Styrene)	Lightweight, low-cost, compatible with FDM; PLA is biodegradable and suitable for indoor use
Weight	≤ 150 grams including camera	Maintains robot balance and avoids toppling during movement
environmental factors	The operation environment should have minimal vibration, low humidity and ambient temperature $\leq 25^\circ\text{C}$.	Indoor environment, $10\text{--}25^\circ\text{C}$, low-humidity atmosphere, running on a plain surface for minimal vibration. PLA is suitable under these conditions
Camera Compatibility	Must fit both Logitech C270 and Intel RealSense D435	Meets coursework requirement for dual-camera integration

DESIGN OF AUTONOMOUS SYSTEMS

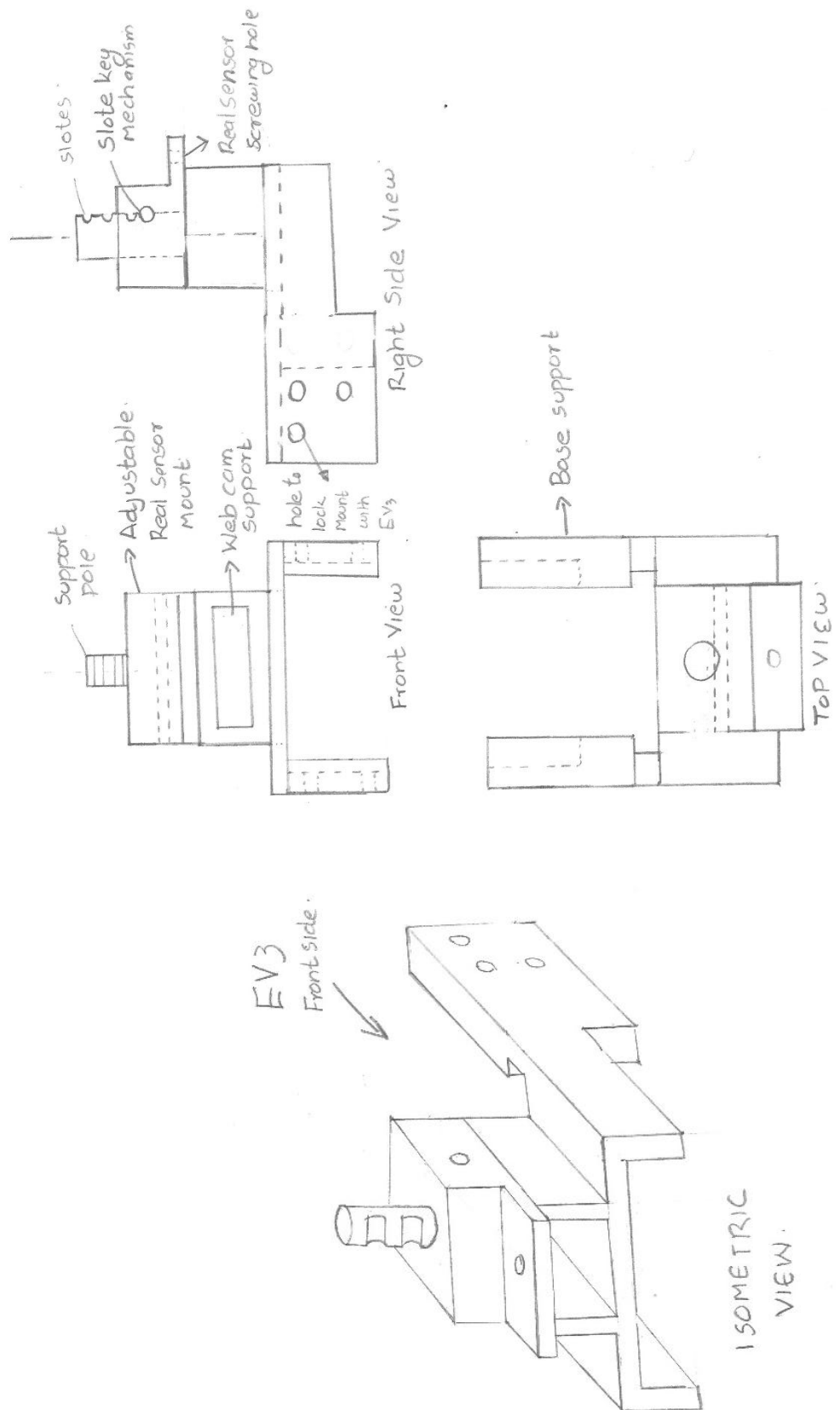
Mounting Method & Ease of Assembly	Snap-fit or Lego-compatible geometry with optional screw points. Assembly time ≤ 2 minutes, no tools required	Ensures quick installation/removal avoids permanent modifications
Field of View	No obstruction within $\pm 40^\circ$ horizontal and vertical viewing angle	Ensures a clear vision for image processing tasks, ergonomics for visibility.
Adjustability	Tilt adjustment of $\pm 30^\circ$ maximum for web-camera and height adjustment up to 3 cm for RealSense D435.	Allows flexible camera orientation for different tasks
Stability	Must remain rigid under robot acceleration and vibration	Critical for reducing image distortion during navigation
Manufacturability	Printable on a standard FDM printer	Supports rapid prototyping and iteration using available lab equipment

5.3 Conceptual Design Generation

For the autonomous vehicle's seamless function, a few conceptual designs were developed that meet the project's defined specifications and functional requirements—such as height adjustability, secure sensor and webcam placement, structural stability, and compatibility with the LEGO EV3 robot platform.

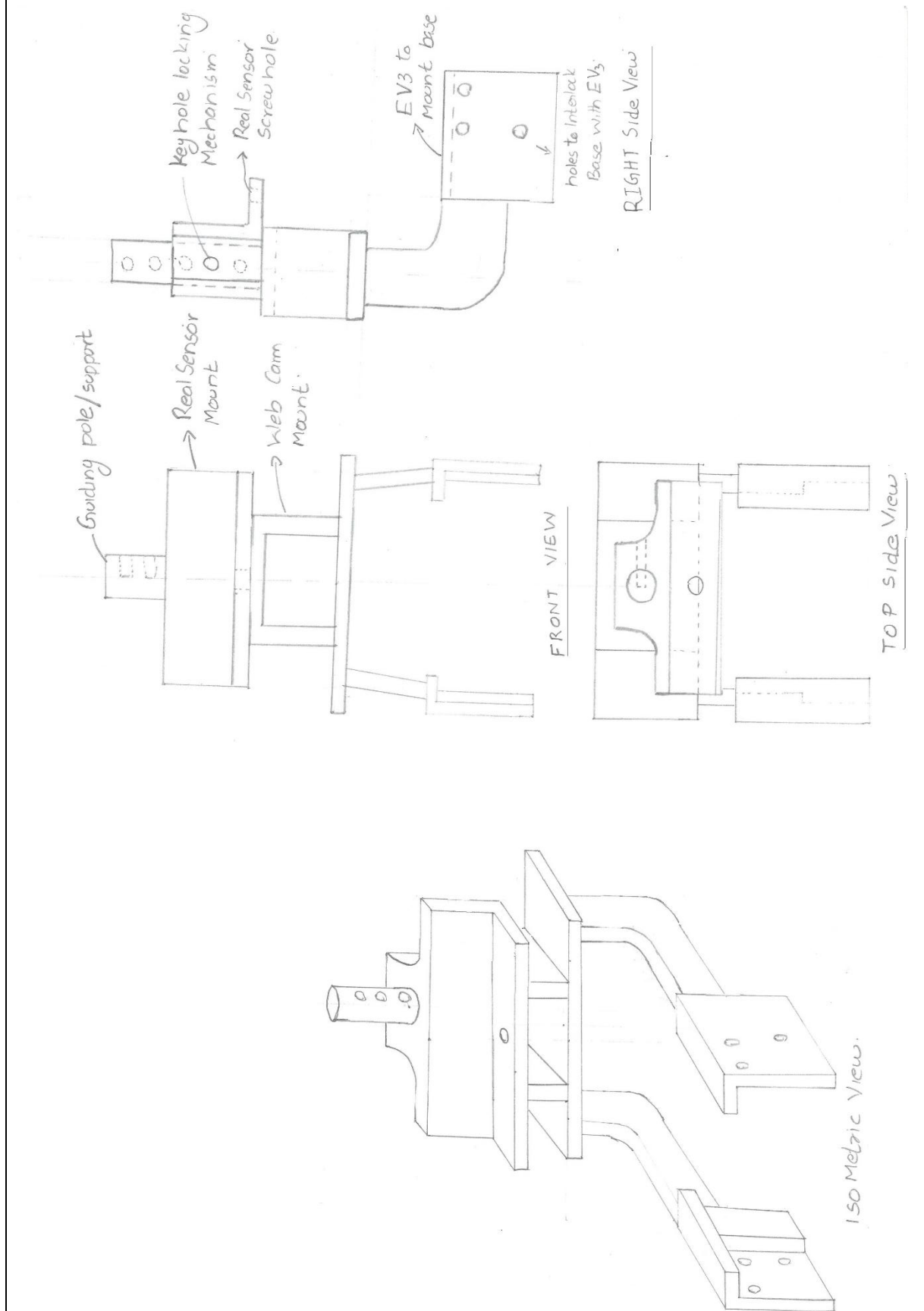
DESIGN OF AUTONOMOUS SYSTEMS

Conceptual Design 1



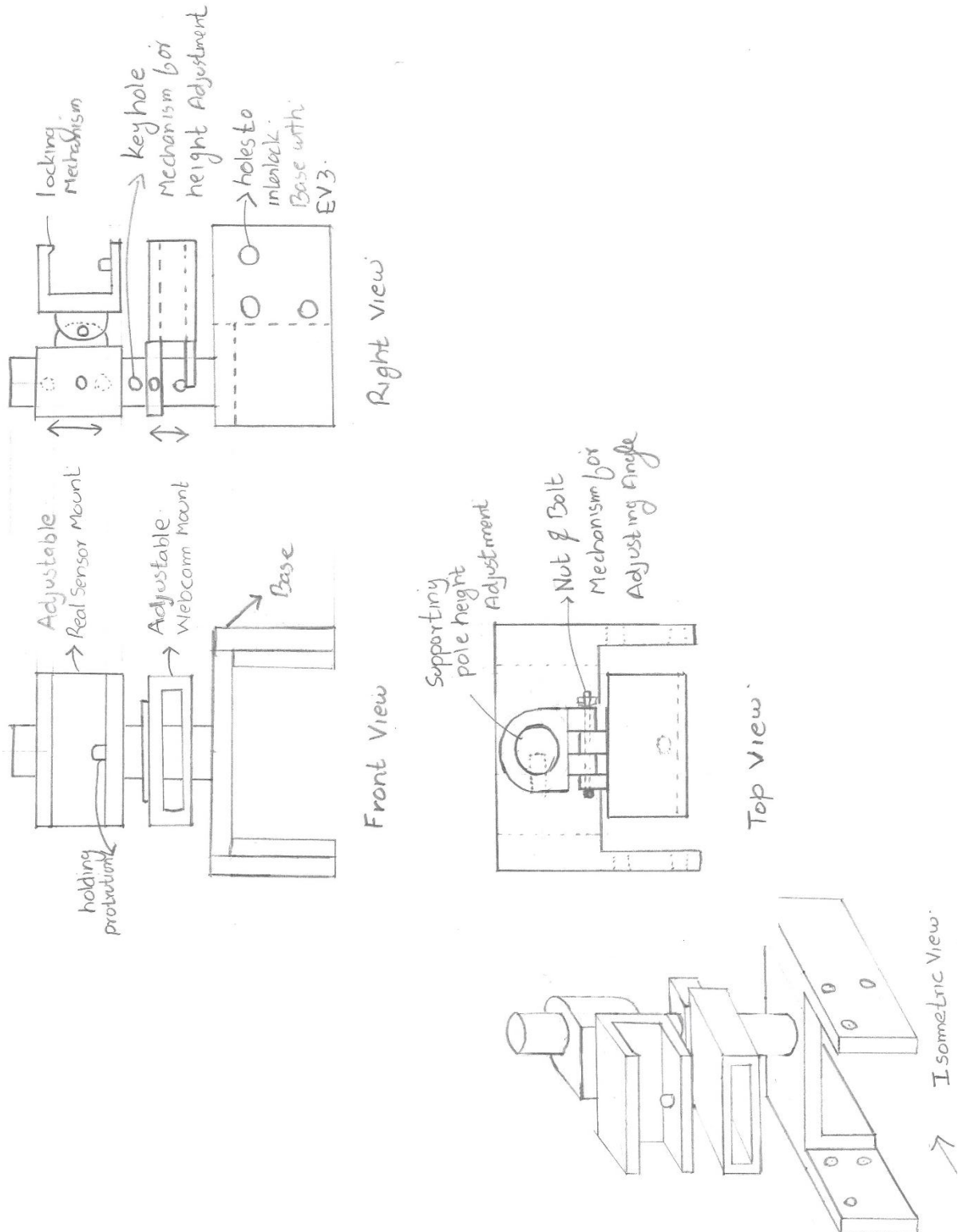
DESIGN OF AUTONOMOUS SYSTEMS

Conceptual Design 2



DESIGN OF AUTONOMOUS SYSTEMS

Conceptual Design 3



DESIGN OF AUTONOMOUS SYSTEMS

Required Specification	Conceptual Design		
	1	2	3
Height Adjustability	Real Sensor Height is adjustable	Real Sensor Height is adjustable	Real Sensor and Web-cam height can be adjusted
Angle Adjustability	The webcam sensor's angle can be adjusted with its inbuilt feature but not with real sensor	The webcam sensor's angle can be adjusted with its built-in feature but not with real sensor	The webcam sensor's angle can be adjusted with its inbuilt feature and real sensor also can be adjusted
Stability on movement of EV3	Stable, however , When sudden stopping of EV3, front mounts shift weight forward, creating torque that lifts the rear tires due to inertia	Stable, the mount is placed on backend of EV3, sudden change of velocity won't affect stability	Stable, the mount is placed on near he end of EV3, sudden change of velocity won't affect stability
Weight	Comparatively Less Weight	Less Weight	Comparatively high Weight
Line of Sight	Comparatively, the line of sight is similar to conceptual design 3 because it also have to consider the traffic signals.	high line of sight, however, has a blind spot 30 cm in front of the Lego EV3 for the webcam	high line of sight, however, has blind spot 20 cm infront of the Lego EV3 for the webcam
Material & Fastening	A PLA pin is used	A PLA pin is used	A combination of M3 x 12mm nut and bolt, M3 x 12mm screw and PLA is used
Ease of assembly	Easy to assemble	Easy to assemble	Comparatively difficult to assemble
Dimension	Mount size $\leq$ 100mm (H) $\times$ 80mm (W) $\times$ 100mm (D)	Mount size $\leq$ 130mm (H) $\times$ 80mm (W) $\times$ 100mm (D)	Mount size $\leq$ 150mm (H) $\times$ 80mm (W) $\times$ 100mm (D)
Environmental Factors	Operatable under smooth surface, low humid and cold temperature	Operatable under smooth surface, low humid and cold temperature	Operatable under smooth surface, low humid and cold temperature
EV3 Compatibility	A portion of the mounts support is on the display of EV3,thus	The mount does not obstruct any part of the EV3 LEGO robot, allowing full access	While the mount does not interfere with the robot's functionality, it does reduce

DESIGN OF AUTONOMOUS SYSTEMS

	obstructing the EV3 display	to all ports, sensors, and mechanical components	accessibility to the EV3 switches due to its position above them
--	-----------------------------	--	--

The three conceptual designs presented offer varied solutions for mounting a camera with height adjustability, stability, and compatibility with the EV3 robot. Each concept demonstrates a unique approach to supporting the webcam and sensor, using different locking and adjustment mechanisms. These designs provide a strong foundation for technical evaluation and prototype development, ensuring the final solution will be both functional and adaptable to project needs.

5.4 Design Evaluation and Selection

Following the generation of multiple conceptual designs for the EV3-compatible camera mount, each concept was critically assessed against the established design specifications (Section 5.3). Key criteria included structural stability, ease of assembly, accessibility of EV3 controls, compatibility with LEGO elements, weight.

To ensure an objective and justifiable selection, a Weighted and Ranked (W&R) matrix was employed. Each design was scored based on its performance across key criteria, with weights assigned according to the relative importance of each specification.

Evaluation Criteria and Weighting

Evaluation Criteria and Weighting	
Criterion	Weight (%)
Height Adjustability	10
Angle Adjustability	10
Stability on movement of EV3	20
Weight	10
Line of Sight	10
Material & Fastening	5
Ease of assembly	10
Dimension	5
Environmental Factors	5
EV3 Compatibility	15

Weighted and Ranked Evaluation Matrix				
Design Criteria	Weight (%)	Conceptual		
		Design 1	Design 2	Design 3
Height Adjustability	10	8 (80)	8 (80)	9 (90)
Angle Adjustability	10	6 (60)	6 (60)	9 (90)
Stability on movement of EV3	20	6 (120)	9 (180)	9 (180)
Weight	10	9 (90)	9 (90)	6 (60)

DESIGN OF AUTONOMOUS SYSTEMS

Line of Sight	10	9 (90)	7 (70)	8 (80)
Material & Fastening	5	8 (40)	8 (40)	7 (35)
Ease of assembly	10	9 (90)	9 (90)	6 (60)
Dimension	5	9 (45)	8 (40)	6 (30)
Environmental Factors	5	9 (45)	9 (45)	9 (45)
EV3 Compatibility	15	5 (75)	10 (150)	7 (105)
Total Score	100	735	845	775

*Scores in parentheses are the weighted contributions for each criterion

Selected Concept: Conceptual design Concept 2

Concept 2 achieved the highest total score of 845, primarily due to its excellent stability, high EV3 compatibility, and balanced performance across all other categories. It avoids any obstruction of the EV3's functional parts, maintains good height and angle adjustability, and is lightweight and easy to assemble. Although it has a minor blind spot in the line of sight, this is deemed acceptable given the overall performance.

5.5 Detailed Design (CAD)

This section presents the detailed CAD design of a custom camera mount for the Lego robot platform, developed as part of the autonomous navigation system. The mount was created using **SolidWorks**, ensuring it is fully constrained, dimensionally accurate, and suitable for 3D printing. The design prioritizes **stability**, **adjustability**, and **unobstructed sensor visibility**.

Group Members	Component	Responsibility
Rahul (K2447273)	EV3 Mounting Interface	Create the keyhole-based connector that interfaces with the EV3 brick, including a mount rests on the EV3 surface to maximise stability and easier connection
Joshua (K2445563)	Webcam Mount & Platform	Design the platform that holds the webcam, ensure secure mounting, and check alignment with EV3's front view.
Omkar (K2441768)	Structural Support Arms & Stability	Design the curved support arms and base structure that distribute load (to reduce stress concentration) and maintain balance during EV3 movement.
Sarath (K2454461)	Real Sensor Mount, Height Support & pin	Model the height-adjustable post and mount for the real sensor, including clearances and mounting holes.

Required Documents

DESIGN OF AUTONOMOUS SYSTEMS

- SolidWorks Files

[SOLID WORKS](#)

- CAD Models

Refer to Appendix 2

[2D-DAS-2025.SLDDRW](#)

- Constrained Drawings

Refer to Appendix 3

5.6 Design Communication

Effective communication of engineering design is essential for the accurate manufacturing, assembly, and validation of components. This section presents a complete set of technical documentation for the custom-designed camera mount, explicitly developed for the Lego-based autonomous navigation platform. All drawings have been produced using SolidWorks and conform to BS 8888, the British Standard for technical product specification.

The design is communicated through detailed 2D technical drawings and 3d CAD models for each part and assembly. These drawings include standard orthographic views (Top, Front, Side), isometric perspectives, section views where required, and complete dimensional annotation. All components are fully constrained and dimensioned to eliminate ambiguity during fabrication.

The assembly drawing includes a comprehensive Bill of Materials (BOM) detailing part numbers, descriptions, and quantities. An exploded view with balloon annotations is also included (Appendix 2-Camera Mount), clearly identifying each part and its position within the final assembly. Clear line types, appropriate symbols, and labelled dimensions comply with BS 8888 and ISO 128 presentation rules, enhancing the legibility and professional standard of the documentation.

Materials such as PLA have been selected based on their widespread use in additive manufacturing and their suitability for lightweight structural applications. These plastics offer easy prototyping while maintaining acceptable mechanical integrity for mounting lightweight sensors.

This documentation ensures that the design can be consistently interpreted and manufactured within a UK engineering context or beyond by following recognised engineering standards such as BS 8888 and ISO 128.

DESIGN OF AUTONOMOUS SYSTEMS

BILL of Materials

Item No.	Part Number	Part Name	Quantity	Material	Drawing Sheet No.	Remarks
1	DAS-2025-001	Real Sense Camera Mount	1	PLA	DAS-2025-001	Camera mount for intel RealSense Depth Camera D435
2	DAS-2025-002	Real Sense Camera Mount Pin	1	PLA	DAS-2025-002	Pin holding Real Sense Camera Mount with EV3 to Webcam Mount
3	DAS-2025-003	EV3 to Webcam Mount	1	PLA	DAS-2025-003	Webcam mount that is attached to the Lego EV3 robot

5.7 Prototype Development

Design for Manufacturing and Assembly (DFMA) Considerations

The camera mount design was refined based on DFMA principles to ensure efficient fabrication and assembly. The number of individual components was minimized. All overhangs were limited to $<45^\circ$ to eliminate the need for excessive support material. Holes and joints were dimensioned with adequate clearances (typically +0.2 to 0.3 mm for pins) to accommodate post-print assembly without force fitting or machining. The geometry was made symmetrical where possible to simplify orientation during assembly.

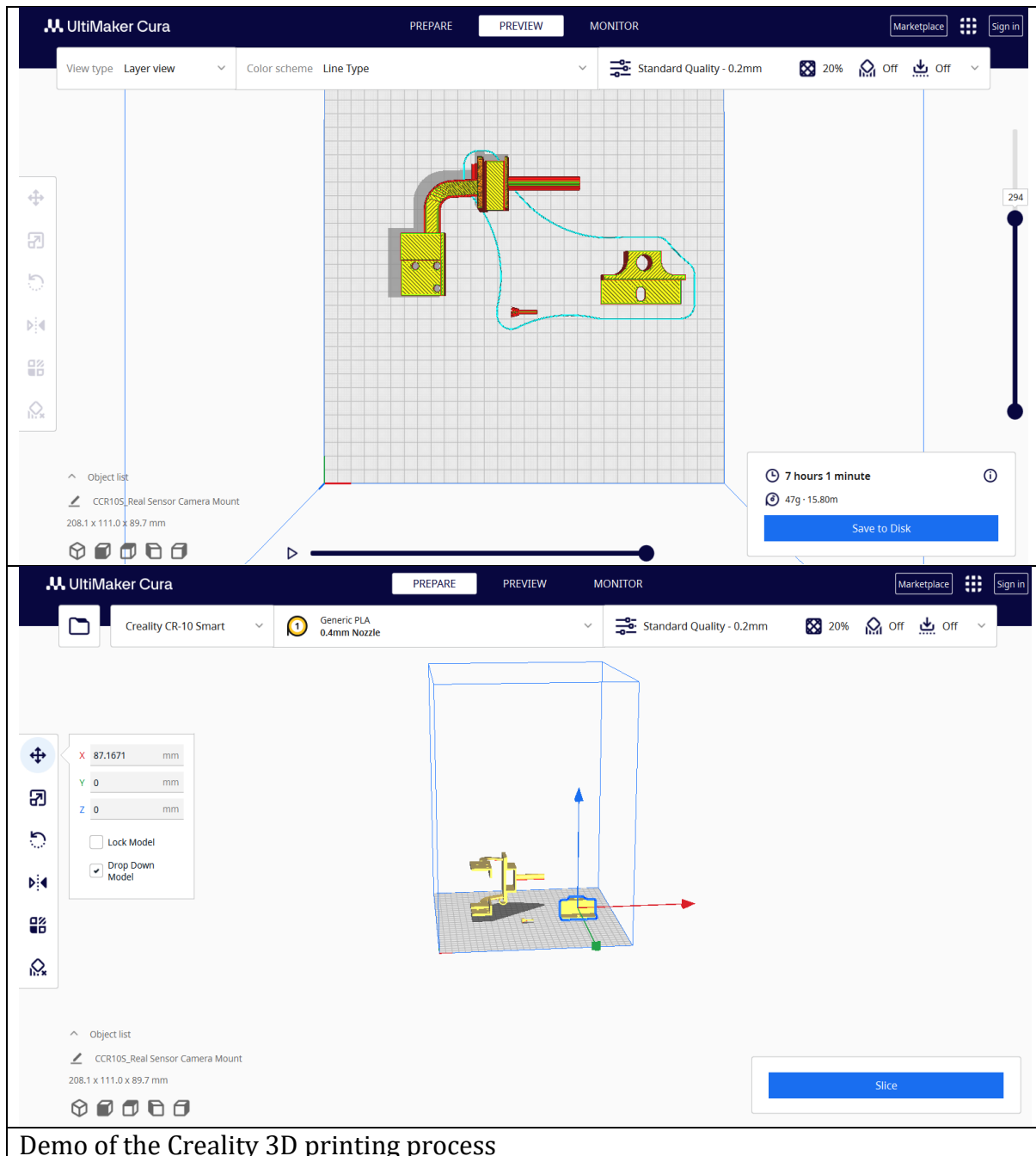
3D Printing Material and Equipment

The prototype components were fabricated using PLA filament due to its ease of printing, dimensional stability, and suitability for rapid prototyping applications. The following 3D printing setup was used:

- Printer Model: Creality
- Material: Generic PLA,
- Wall Thickness: 2 perimeters (approx. 0.8 mm)
- Supports: None required (geometry DFMA-optimised)
- Slicer Software: Ultimaker Cura

DESIGN OF AUTONOMOUS SYSTEMS

Each part was sliced and printed individually with appropriate orientation to minimise warping and maximise surface quality on functional faces.

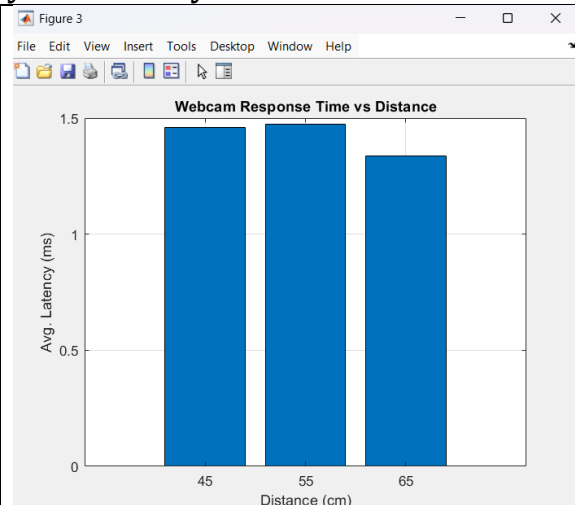
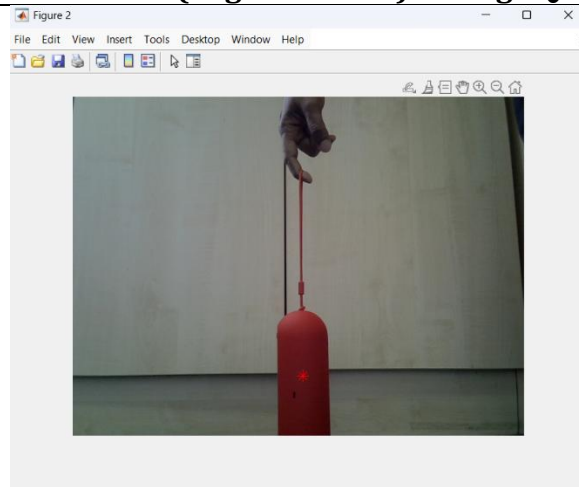


Demo of the Creality 3D printing process

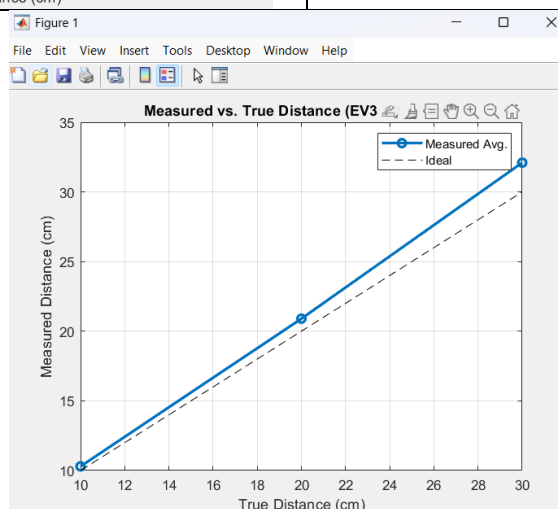
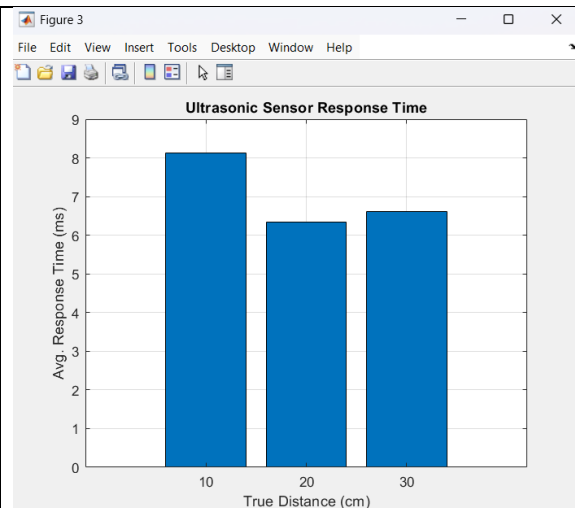
DESIGN OF AUTONOMOUS SYSTEMS

Appendix 1, Sensor characterisation

1. Webcam (Logitech C270) - Image Quality and Latency

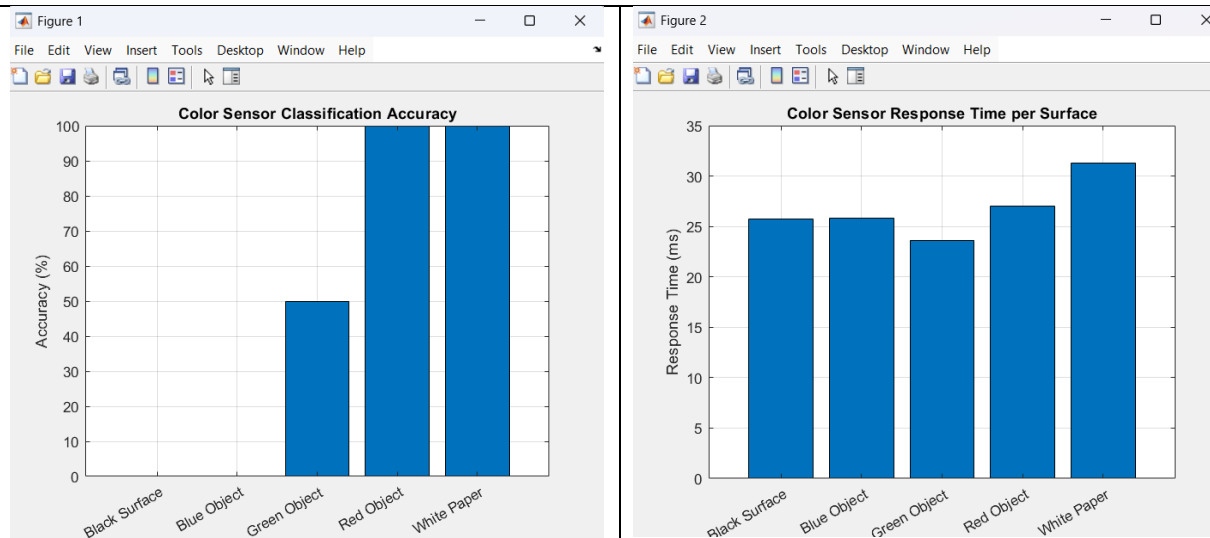


2. Ultrasonic Sensor - Distance vs. Accuracy

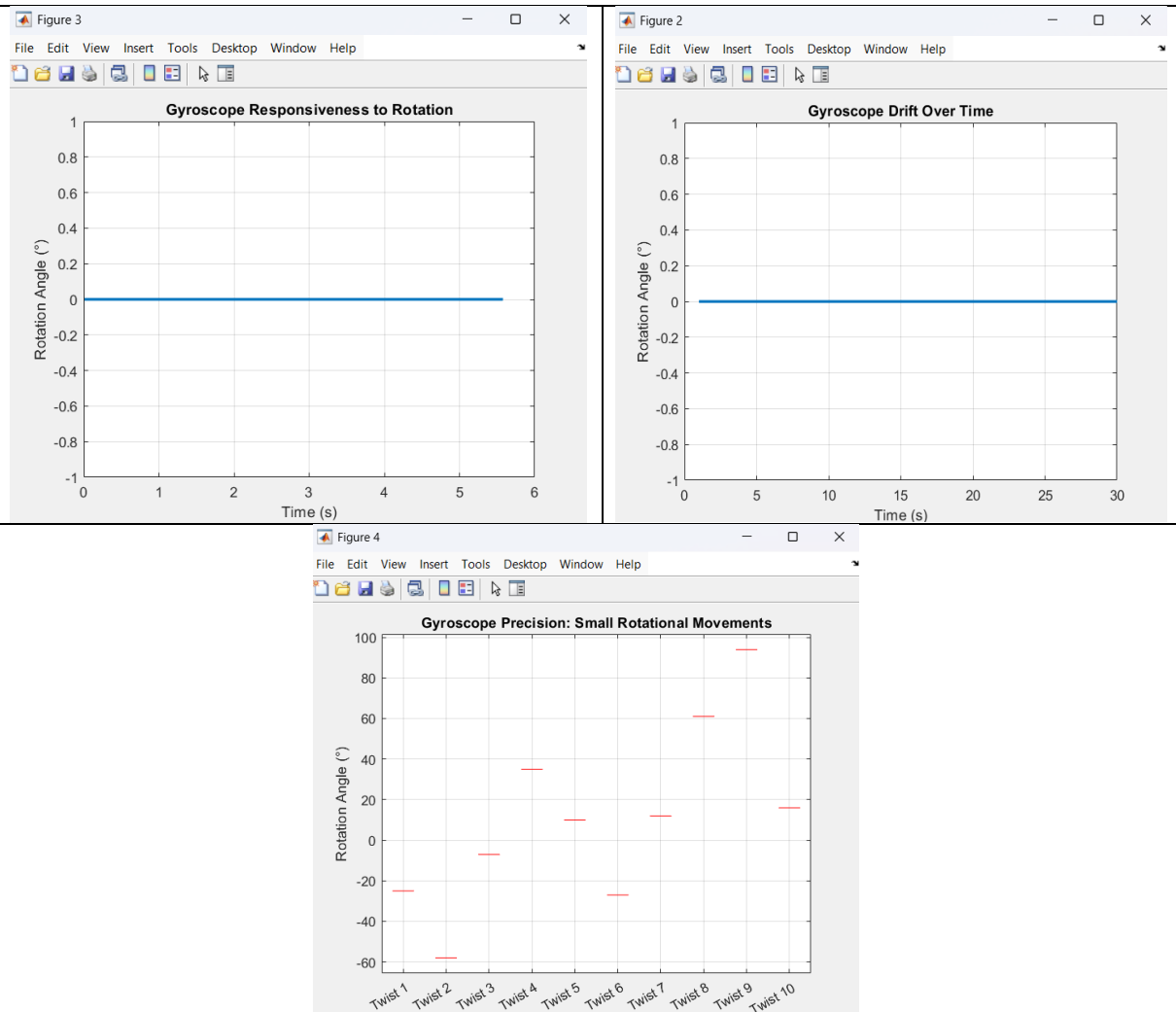


DESIGN OF AUTONOMOUS SYSTEMS

3. Colour sensor-RGB Recognition & Surface Response

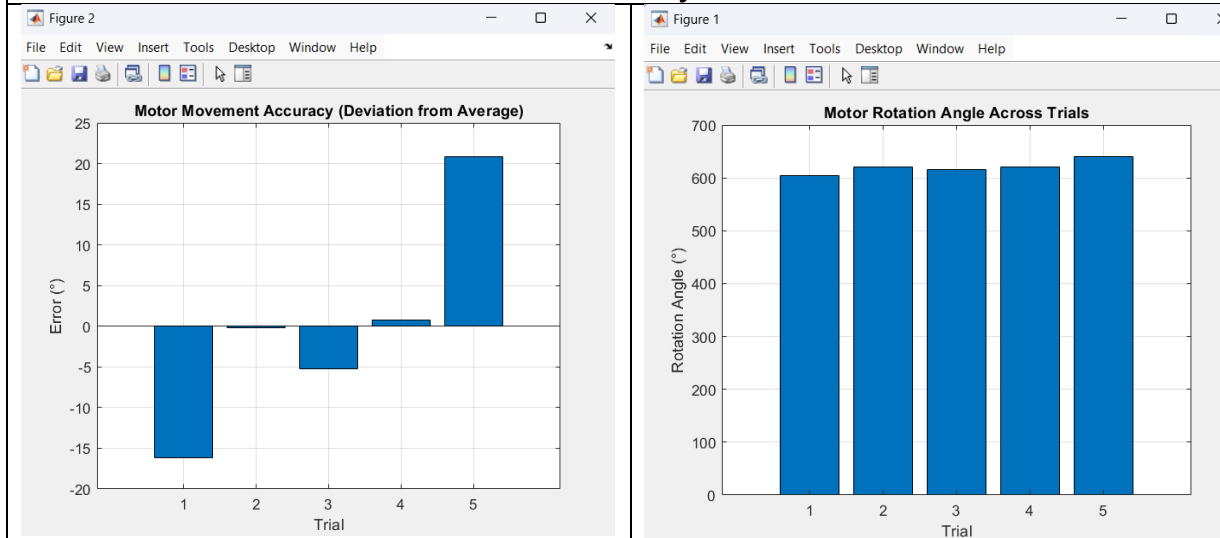


4. Gyroscope - Drift & Responsiveness



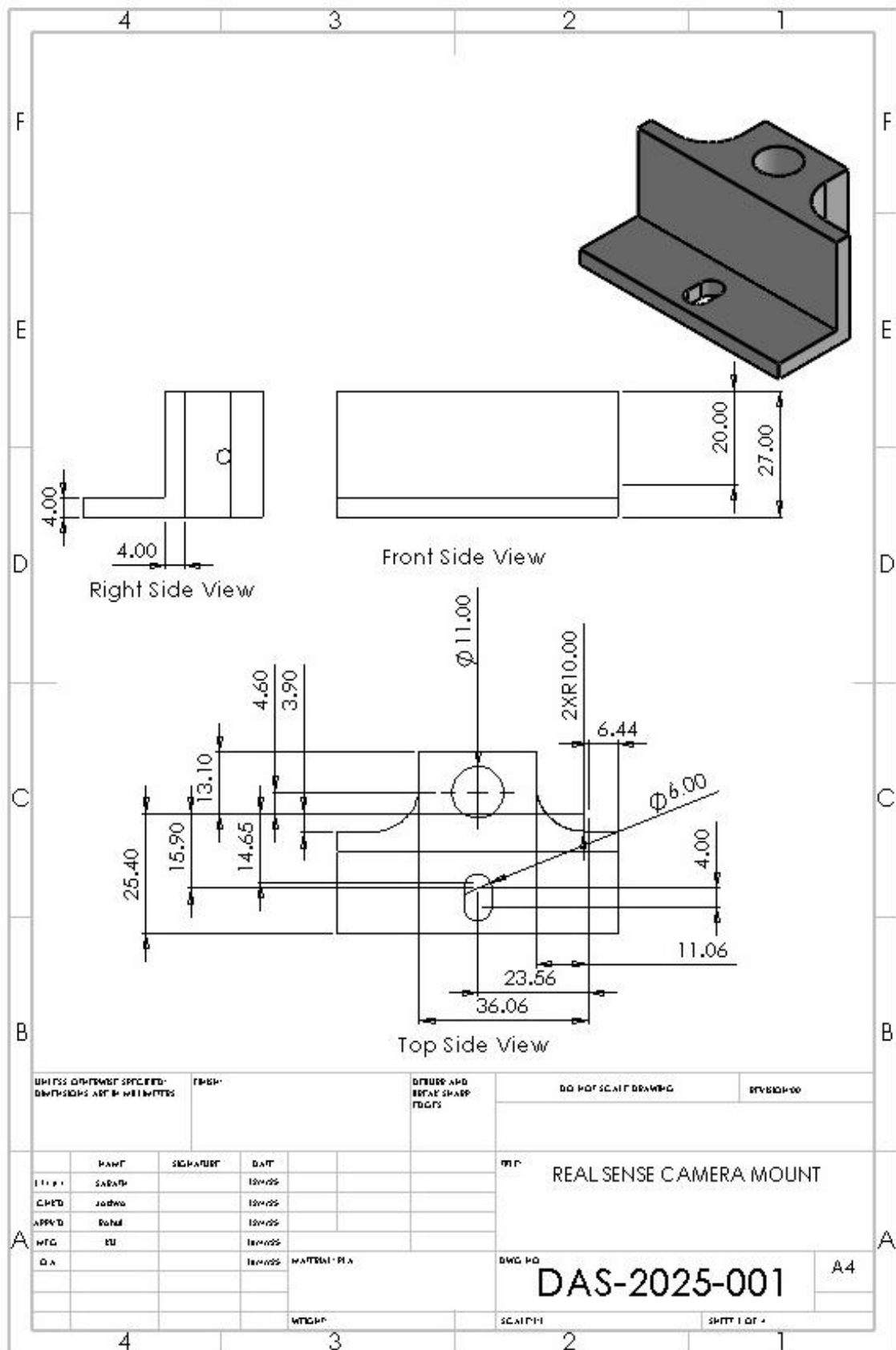
DESIGN OF AUTONOMOUS SYSTEMS

5. Motors and Encoders – Movement Accuracy



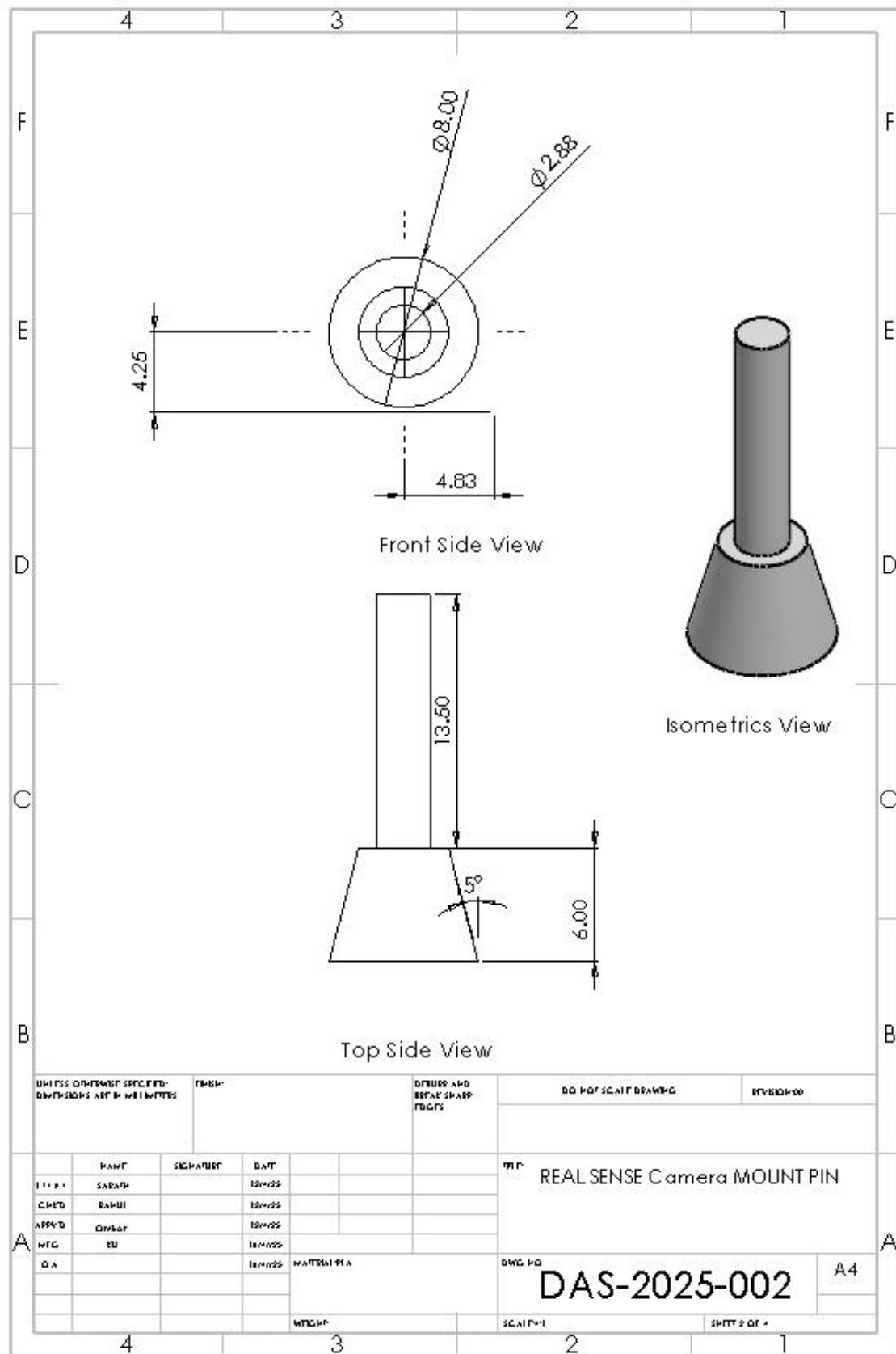
DESIGN OF AUTONOMOUS SYSTEMS

Appendix 2, CAD Models



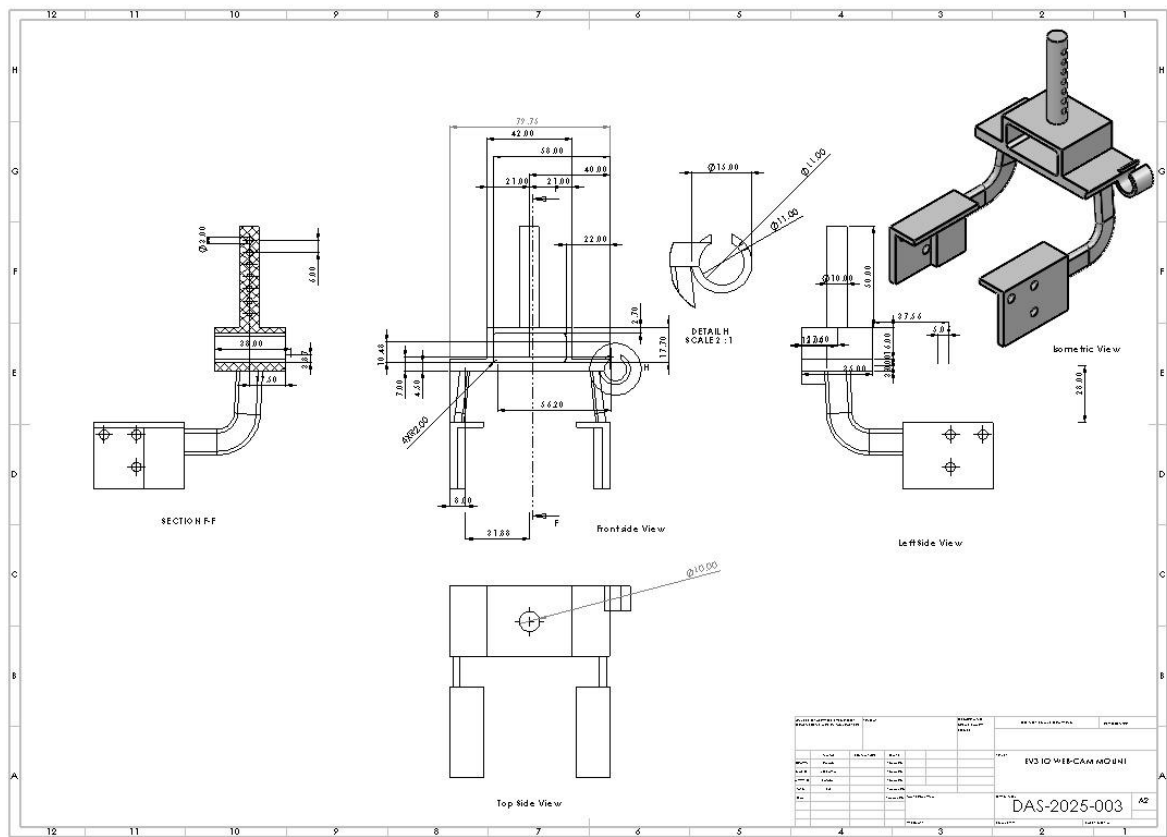
Part 1- Real Sens Camera Mount – CAD Model In 2D and Isometric View

DESIGN OF AUTONOMOUS SYSTEMS



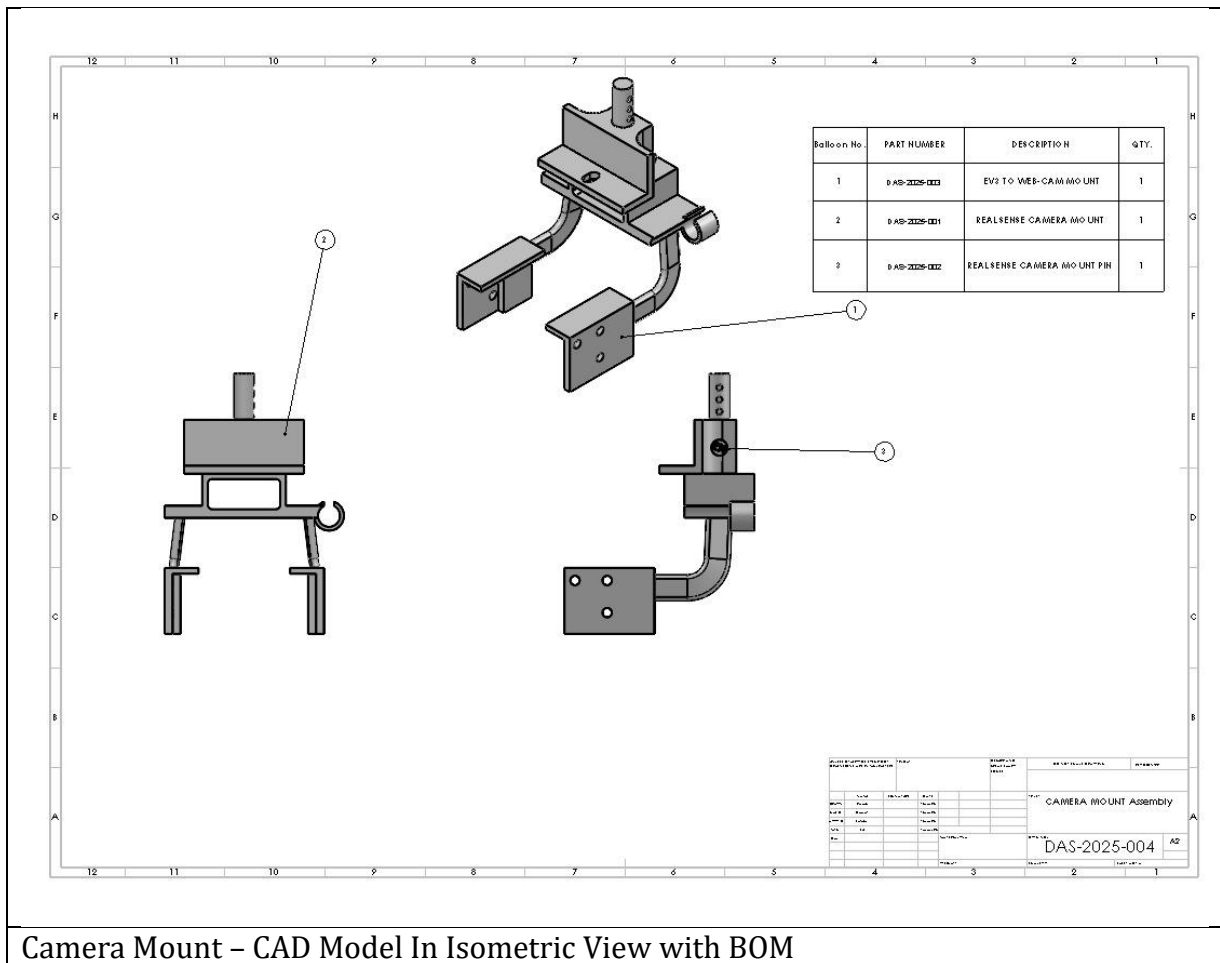
Part 2-Real Sens Camera Mount Pin- CAD Model In 2D and Isometric View

DESIGN OF AUTONOMOUS SYSTEMS



Part 3-EV3 to Web-Cam Mount – CAD Model In 2D and Isometric View

DESIGN OF AUTONOMOUS SYSTEMS

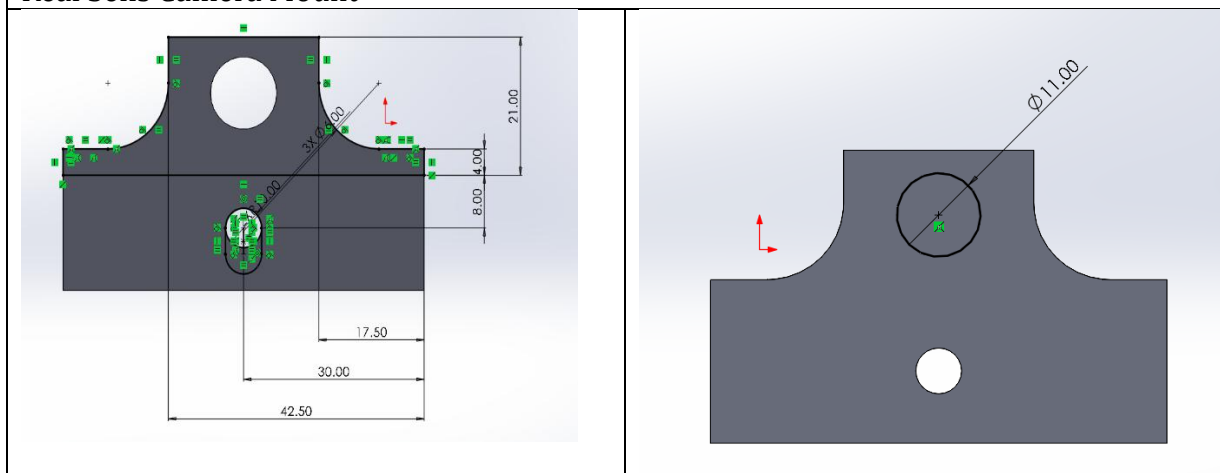


Camera Mount – CAD Model In Isometric View with BOM

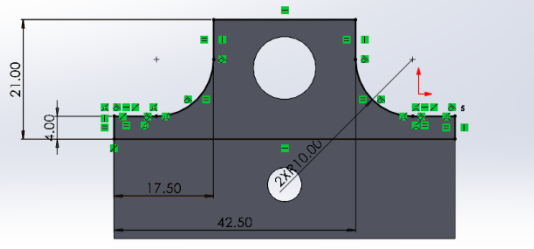
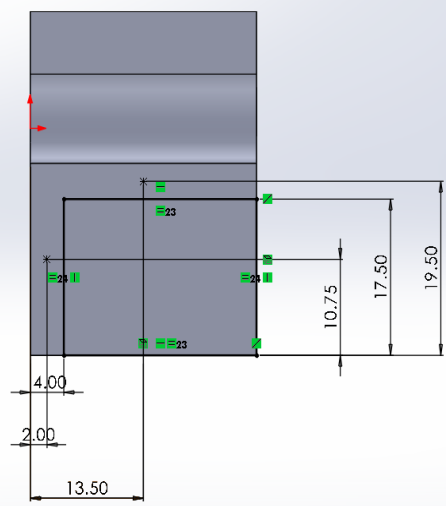
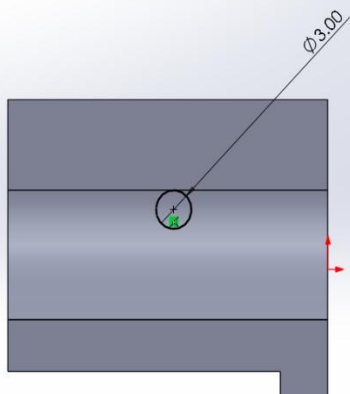
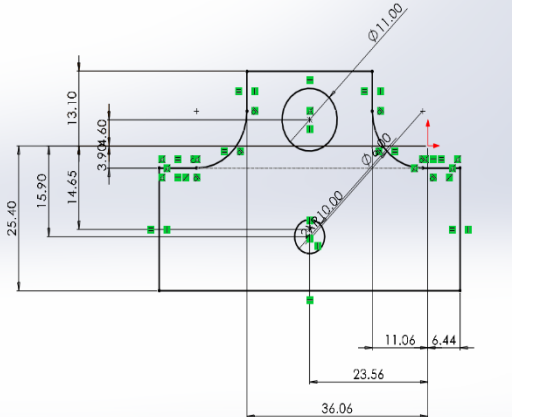
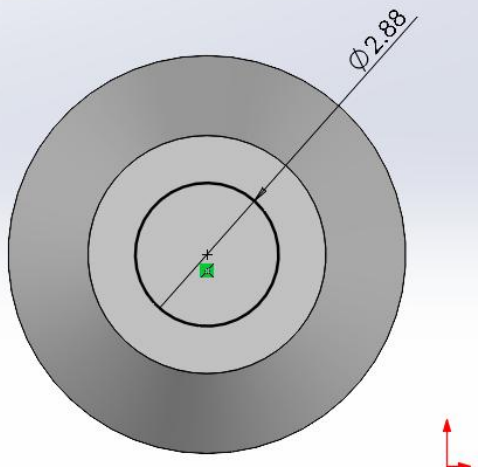
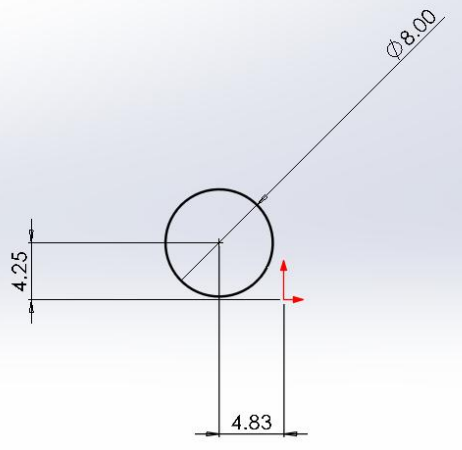
Note: All dimensions are in mm

Appendix 3, Constrained Models

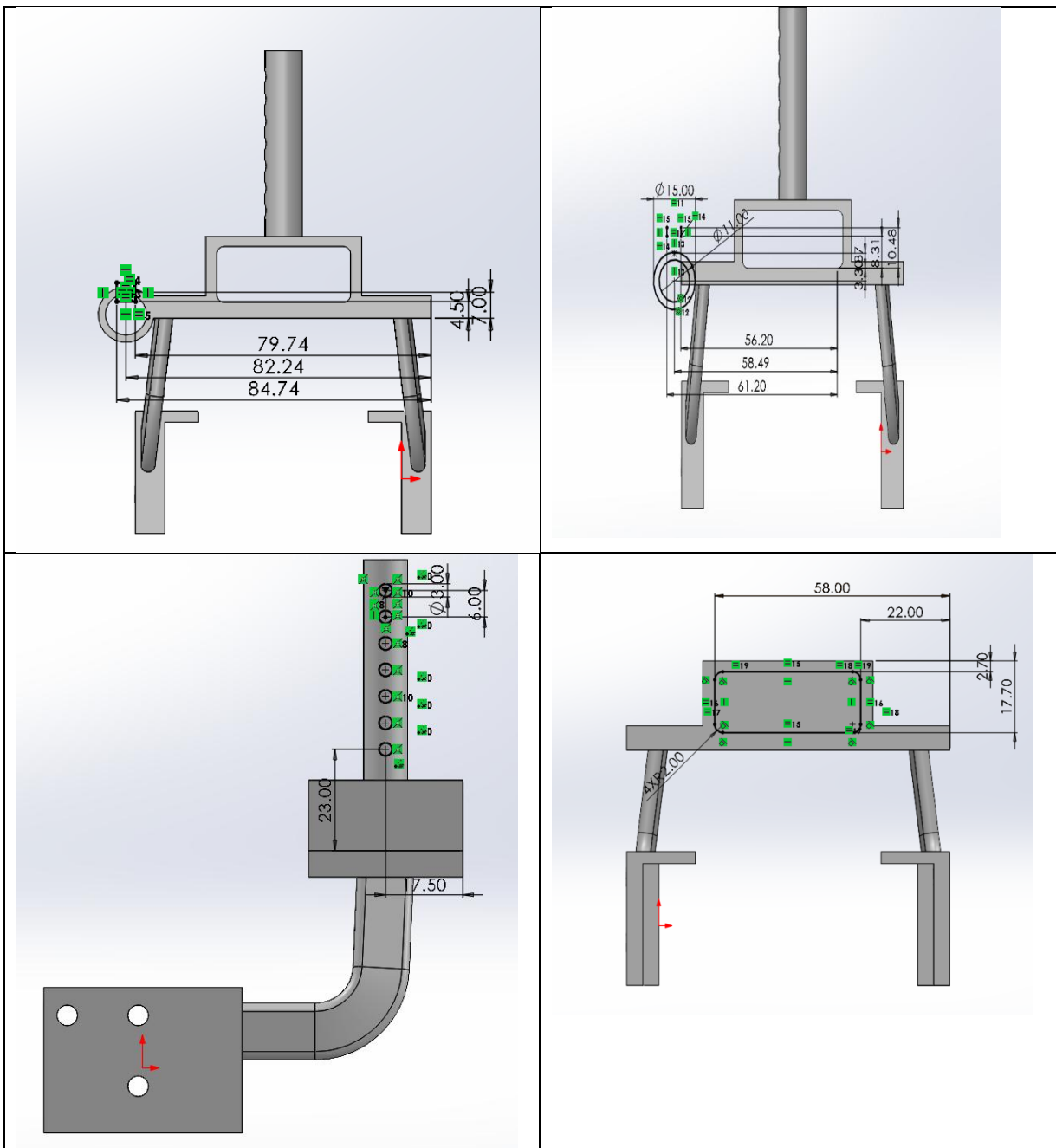
Real Sens Camera Mount



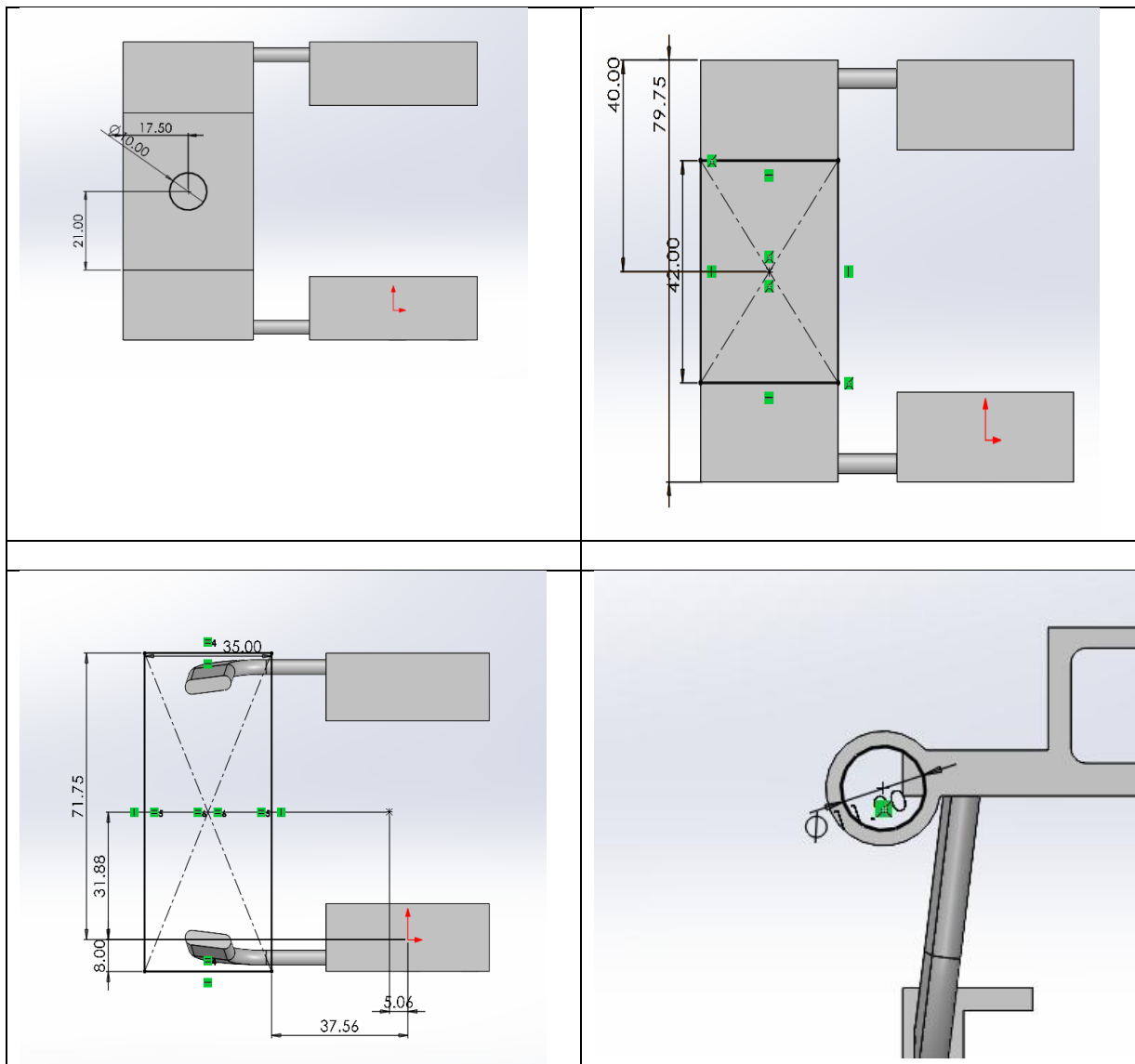
DESIGN OF AUTONOMOUS SYSTEMS

	
	
Real Sens Camera Mount Pin	
	
EV3 to Web-Cam Mount	

DESIGN OF AUTONOMOUS SYSTEMS

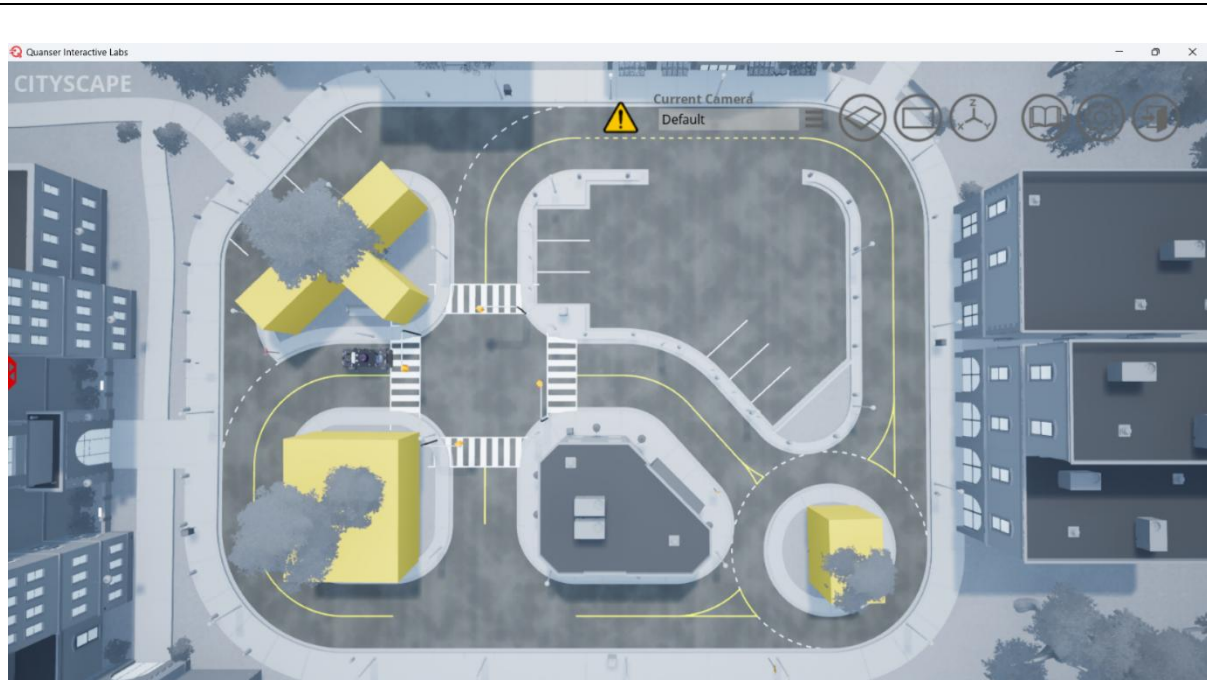


DESIGN OF AUTONOMOUS SYSTEMS



DESIGN OF AUTONOMOUS SYSTEMS

Appendix 4, Quencer Virtual Simulation Lab and Simulink

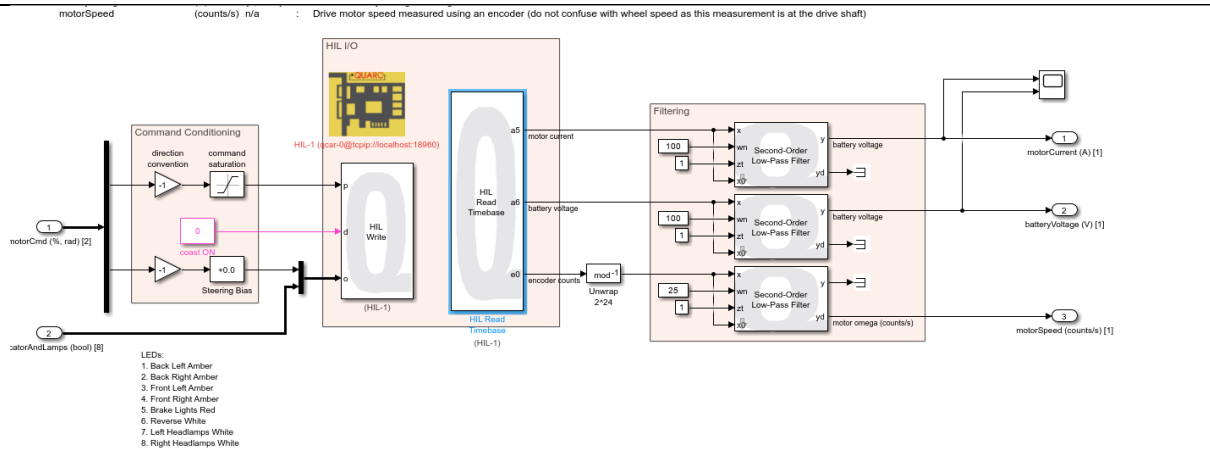


DESIGN OF AUTONOMOUS SYSTEMS



DESIGN OF AUTONOMOUS SYSTEMS

Simulink



basicQCarIO Module:

This subsystem provides the core and basic I/O capabilities to read data from and write data to the QCar platform. The HIL Initialize block configures the IO settings, and the HIL Write and Read blocks are used for the actual IO. You can also monitor sample and communication times.

Inputs:

motorCmd (% rad) n/a : Vector input for the motor drive command (percentage throttle) as well as steering command (in rad)

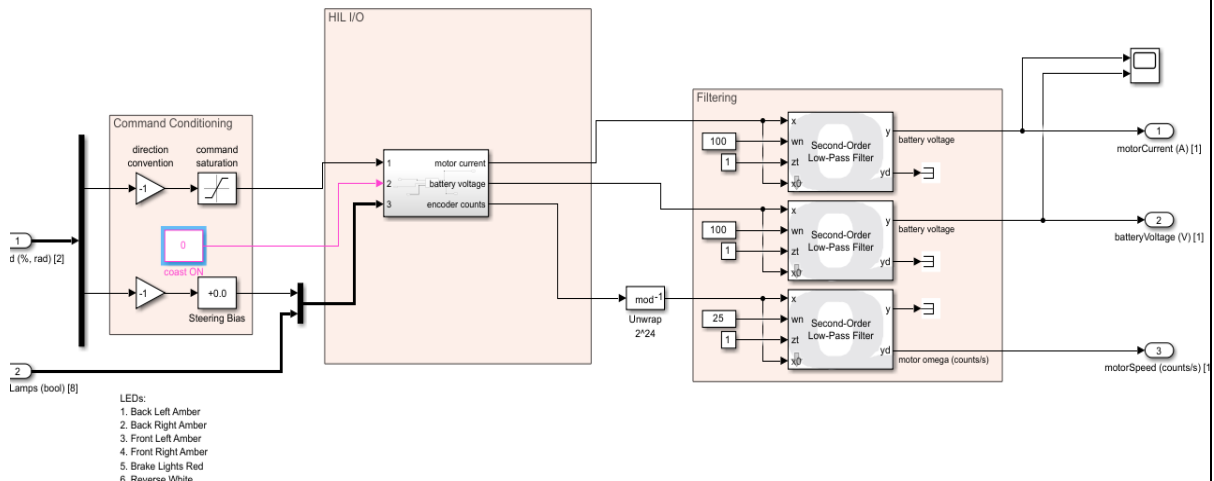
indicatorsAndLamps (%) {0,1} : Percentage brightness command for the 8 LEDs on the QCar platform.

Outputs:

motorCurrent (A) n/a : Motor current being pulled by the main drive motor

batteryVoltage (V) {0, 12.6} : Current battery voltage reading

motorSpeed (counts/s) n/a : Drive motor speed measured using an encoder (do not confuse with wheel speed as this measurement is at the drive shaft)



DESIGN OF AUTONOMOUS SYSTEMS

References

- LEGO Group (n.d.) LEGO MINDSTORMS EV3 User Guide – Tablet Edition. [PDF] Available at:
https://www.lego.com/cdn/cs/set/assets/bltbef4d6ce0f40363c/LMSUser_Guide_LEG_O_MINDSTORMS_EV3_11_Tablet_ENUS.pdf [Accessed 8 May 2025].
- Quanser Inc. (n.d.) *QCar User Manual – System Hardware*. [PDF] Available at: [internal document or URL if available] [Accessed 11 April 2025]
[Qcar user manual system hardware.pdf](#)
- LEGO Education (n.d.) *EV3 User Guide (US Edition)*. [PDF] Available at:
https://assets.education.lego.com/v3/assets/blt293eea581807678a/bltc5b2d4e8f6b6ccdc/5f8806d2f6a0a50f825b03da/ev3_user_guide_us.pdf [Accessed 27 April 2025]
- LEGO Education (n.d.) *Using Sensors – Colour Sensor*. [online] Available at:
https://ev3-help-online.api.education.lego.com/Education/en-gb/page.html?Path=editor%2FUsingSensors_Color.html [Accessed 18 April 2025]
- Raut, A., Padole, A., Jaiswal, R., Mahajan, S. and Patil, V. (2024) *Autonomous Vehicle using Raspberry Pi and YOLO Algorithm*. *International Research Journal of Engineering and Technology (IRJET)*, 11(3), pp. 653–656. [PDF] Available at:
<https://www.irjet.net/archives/V11/I3/IRJET-V11I398.pdf> [Accessed 3 April 2025]
- MathWorks (n.d.) *Overview of Lane Following Systems*. [online] Available at:
<https://uk.mathworks.com/help/driving/ug/overview-of-lane-following-systems.html> [Accessed 2 May 2025].
- MathWorks (n.d.) *Edge Detection*. [online] Available at:
<https://uk.mathworks.com/discovery/edge-detection.html> [Accessed 4 May 2025].
- MathWorks (n.d.) *Image Segmentation Using the Colour Thresholder App*. [online] Available at: <https://uk.mathworks.com/help/images/image-segmentation-using-the-color-thesholder-app.html> [Accessed 11 April 2025].
- MathWorks (n.d.) *hough*. [online] Available at:
<https://uk.mathworks.com/help/images/ref/hough.html> [Accessed 18 April 2025].
- MathWorks (n.d.) *Lane Following Control with Sensor Fusion and Lane Detection*. [online] Available at: <https://uk.mathworks.com/help/driving/ug/lane-following-control-with-sensor-fusion-and-lane-detection.html> [Accessed 27 April 2025]
- MathWorks (2022). *Creating a colour mask for an image in HSV*. [online] MATLAB Central. Available at: <https://uk.mathworks.com/matlabcentral/answers/1771110-creating-a-colour-mask-for-an-image-in-hsv> [Accessed 3 April 2025].

DESIGN OF AUTONOMOUS SYSTEMS

- MathWorks (n.d.) *Colour Thresholder App*. [online] Available at: <https://uk.mathworks.com/help/images/ref/colorthresholder-app.html> [Accessed 11 April 2025]
- MathWorks (n.d.) *Morphological Filtering*. [online] Available at: <https://uk.mathworks.com/help/images/morphological-filtering.html> [Accessed 18 April 2025]
- MathWorks (n.d.) *adapthisteq*. [online] Available at: <https://uk.mathworks.com/help/images/ref/adapthisteq.html> [Accessed 2 May 2025].
- MathWorks (n.d.) *regionprops*. [online] Available at: <https://uk.mathworks.com/help/images/ref/regionprops.html> [Accessed 4 May 2025]
- MathWorks (n.d.) *Object Detection Using YOLO v4 Deep Learning*. [online] Available at: <https://uk.mathworks.com/help/vision/ug/object-detection-using-yolov4-deep-learning.html> [Accessed 27 April 2025].
- MathWorks (n.d.) *googlenet*. [online] Available at: <https://www.mathworks.com/help/releases/R2021a/deeplearning/ref/googlenet.html> [Accessed 3 April 2025].
- Kaggle (n.d.) *Traffic Lights – Datasets Search*. [online] Available at: <https://www.kaggle.com/search?q=traffic+lights+in%3Adatasets> [Accessed 11 April 2025].
- Roboflow (n.d.) *Traffic and Road Signs*. [online] Available at: <https://universe.roboflow.com/usmanchaudhry622-gmail-com/traffic-and-road-signs> [Accessed 18 April 2025].
- MathWorks (n.d.) *Define Custom Deep Learning Layers*. [online] Available at: <https://uk.mathworks.com/help/deeplearning/ug/define-custom-deep-learning-layers.html> [Accessed 27 April 2025].
- MathWorks (n.d.) *Image Augmentation Using Image Processing Toolbox*. [online] Available at: <https://uk.mathworks.com/help/deeplearning/ug/image-augmentation-using-image-processing-toolbox.html> [Accessed 2 May 2025].
- MathWorks (n.d.) *Tune Experiment Hyperparameters Using Bayesian Optimization*. [online] Available at: <https://uk.mathworks.com/help/deeplearning/ug/tune-experiment-hyperparameters-using-bayesian-optimization.html> [Accessed 4 May 2025].
- MathWorks (n.d.) *Experiment Manager App*. [online] Available at: <https://uk.mathworks.com/help/deeplearning/ref/experimentmanager-app.html> [Accessed 3 April 2025]
- MathWorks (n.d.) *mobilenetv2*. [online] Available at: <https://uk.mathworks.com/help/deeplearning/ref/mobilenetv2.html> [Accessed 11 April 2025].

DESIGN OF AUTONOMOUS SYSTEMS

- MathWorks (n.d.) *Fuzzy Logic Controller*. [online] Available at: <http://uk.mathworks.com/help/fuzzy/fuzzylogiccontroller.html> [Accessed 18 April 2025]
- MathWorks (n.d.) *Kalman Filtering*. [online] Available at: <https://uk.mathworks.com/help/control/ug/kalman-filtering.html> [Accessed 27 April 2025].
- GrabCAD (n.d.) *Logitech C270 Webcam*. [online] Available at: <https://grabcad.com/library/logitech-c270-webcam-1> [Accessed 11 April 2025].
- GrabCAD (n.d.) *Intel RealSense D435*. [online] Available at: <https://grabcad.com/library/intel-realsense-d435-2> [Accessed 18 April 2025].
- Kingston University (n.d.) *Design Process*. [online] Available at: <https://canvas.kingston.ac.uk/courses/31281/pages/design-process> [Accessed 3 April 2025].
- Kingston University (n.d.) *Design Process*. [online] Available at: <https://canvas.kingston.ac.uk/courses/31281/pages/design-process> [Accessed 3 April 2025].